

Valkey / Sentinel — Operator Manual

What these technologies do

Valkey stores ChirpStack's fast shared runtime/cache state. Sentinel monitors the Valkey nodes, elects/promotes a writable primary after an approved failure, and tells the HA routing layer which member is primary.

The deployment uses three Valkey data nodes plus three Sentinels. Sentinel quorum is 2.

Current application path

```
ChirpStack
-> valkey.internal.lorawan.com:16379
-> local HAProxy writable-primary route
-> current Sentinel-elected Valkey primary :6379
```

The current primary can change. Do not force a fallback simply to match a dated commissioning note.

Step 1 — Discover service names instead of guessing

Run on each ULC host:

```
for s in valkey redis valkey-server redis-server; do
  systemctl is-active "$s" >/dev/null 2>&1 && echo "data-service=$s"
done
for s in valkey-sentinel redis-sentinel sentinel; do
  systemctl is-active "$s" >/dev/null 2>&1 && echo "sentinel-service=$s"
done
```

If the service is containerized on a future build, inspect the current runtime definition before changing anything.

Step 2 — Check listeners

```
sudo ss -lntp | grep -E ':(6379|26379|16379)\b' || true
```

Expected conceptual ownership:

- Valkey data service: 6379 on intended private address;
- Sentinel: 26379 on intended private address;
- HAProxy writable-primary application endpoint: 16379.

Step 3 — Ask each Valkey data node for its role

Use the protected CA and authentication secret already provisioned for the service. Do not place the secret directly in documentation/history.

Command shape:

```
valkey-cli --tls --cacert <VALKEY_CA> \  
-h <VALKEY_PRIVATE_IP> -p 6379 \  
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE
```

PASS means: exactly one data node reports primary/master in normal state and the remaining expected nodes report replica/slave relationships consistent with it.

Step 4 — Check Sentinel quorum and elected primary

Against each Sentinel:

```
valkey-cli --tls --cacert <VALKEY_CA> \  
-h <SENTINEL_PRIVATE_IP> -p 26379 \  
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \  
SENTINEL CKQUORUM lorawan-valkey
```

Then:

```
valkey-cli --tls --cacert <VALKEY_CA> \  
-h <SENTINEL_PRIVATE_IP> -p 26379 \  
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \  
SENTINEL GET-MASTER-ADDR-BY-NAME lorawan-valkey
```

PASS means: quorum succeeds and Sentinels agree on the current primary address.

Step 5 — Verify the application HA endpoint reaches the writable primary

Using the protected application secret:

```
valkey-cli --tls --cacert <VALKEY_CA> \  
--sni valkey.internal.lorawan.com \  
-h <LOCAL_HAPROXY_PRIVATE_IP> -p 16379 \  
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE
```

PASS means: the stable application endpoint reports primary/master.

If direct nodes are healthy but :16379 reaches a replica, fix HAProxy/Sentinel role selection before restarting ChirpStack.

Step 6 — Check ChirpStack only after the HA endpoint passes

If Valkey is healthy and :16379 is correct, inspect ChirpStack logs for Valkey reconnect/auth/TLS errors. Do not point ChirpStack directly at a specific Valkey member as a workaround.

Never run casually

```
FLUSHALL  
FLUSHDB  
SENTINEL FAILOVER  
manual replica promotion  
mass key deletion
```

These are data/failover operations. Use them only under an explicit test, clean-reset, or recovery procedure with the application writers controlled.

Safe maintenance rule

For one Valkey member:

1. establish the current primary from Sentinel/live ROLE;
2. verify the other members and Sentinel quorum;
3. determine whether the target is primary or replica;
4. if maintenance requires a role change, use the dedicated failover procedure rather than improvising;
5. maintain only the intended member;
6. verify it rejoins in the expected role;
7. verify Sentinel quorum;
8. verify :16379 reports writable primary;
9. verify ChirpStack operation.

Do not repeatedly trigger failover to chase a preferred primary host.

Troubleshooting map

Symptom	First check
ChirpStack says Valkey unavailable	:16379 listener/TLS/role
:16379 returns replica	HAProxy/Sentinel primary discovery
Sentinels disagree	Sentinel connectivity/config/epoch/quorum
one replica stale	replication/network/disk/process
no writable primary	quorum/failover state; avoid manual split-brain
auth error on only one client	client secret/ACL/TLS identity

Use bounded service logs after role checks, not before.

Backup/recovery note

Valkey is not the durable telemetry authority, but its runtime state matters for ChirpStack behavior. Preserve the commissioned persistence/replication settings. Do not restore an arbitrary old RDB over a healthy active cluster without the dedicated recovery procedure.

Completion checklist

- all intended data services reachable;
- exactly one writable primary;
- expected replicas attached;
- all intended Sentinels reachable;
- Sentinel quorum passes;

- Sentinels agree on primary;
- HAProxy :16379 reaches that writable primary;
- ChirpStack can use the stable endpoint.

Detailed runbook

- [../server/cloud-production/08-mqtt-and-valkey.md](#)
- [../server/cloud-production/15-failover-chaos-and-acceptance-testing.md](#)

Supporting runbook: 08-mqtt-and-valkey.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

8. MQTT and Valkey HA on the Three-Node POC

Status: CORE SERVICE LAYER COMPLETE / VALIDATED. Mosquitto 2.0.18 is commissioned on `u1c-01` and `u1c-02` with TLS backend listeners on `:8884`, and the validated MQTT HAProxy TLS passthrough endpoint is `10.104.0.2:8883` using broker certificate identity `mqtt.internal.lorawan.com`; broker-backend failure and recovery have passed. Valkey 7.2.13 is commissioned TLS-only on all three nodes with authenticated replication, three TLS Sentinels (quorum 2), a least-privilege HAProxy health identity, dual writable-primary HAProxy endpoints `10.104.0.2:16379` and `10.104.0.4:16379`, and automatic promotion/rejoin testing. The most recent **Phase 8C.14 commissioning failover checkpoint** elected `u1c-02 (10.104.0.4)` as Valkey primary. Treat that as dated acceptance evidence, not a claim about today's elected primary; do not manually fail back solely to match documentation. ChirpStack workload MQTT authentication/ACLs and the second application-node MQTT routing boundary are intentionally deferred to Phase 9 before first ChirpStack start.

Execution-record authority: sections 8.1-8.17 preserve the original target design and are useful for rationale, but they contain placeholders and assumptions that were superseded during live commissioning. When those planning sections differ from the observed deployment, sections 8.18-8.21 and the live-state summary above are authoritative. In particular, the commissioned Valkey TLS identity is `valkey.internal.lorawan.com`, not `valkey-ha.internal.<DOMAIN>`, and the commissioned MQTT path is currently `10.104.0.2:8883 -> Mosquitto :8884`, not the uncommissioned `mqtt-ha.internal.<DOMAIN>:18883` design.

This manual deploys the two shared runtime dependencies used by ChirpStack:

```
MQTT transport      -> Mosquitto-1 preferred / Mosquitto-2 backup
fast shared state   -> Valkey primary + 2 replicas, managed by 3 Sentinels
```

The physical gateway's local Mosquitto queue remains separate on the Raspberry Pi. Do not confuse that edge buffer with the two cloud brokers.

8.1 Port plan and why the broker backend uses 8884

Use this exact separation:

```
PUBLIC MQTT SERVICE
mqtt.<DOMAIN>:8883
-> DigitalOcean Reserved IPv4
-> current owner: HAProxy on ha-01 OR ha-02 anchor :8883
-> Mosquitto backend TLS :8884

PRIVATE MQTT SERVICE
mqtt-ha.internal.<DOMAIN>:18883
-> local HAProxy on ha-01/ha-02/ha-03
-> same Mosquitto backend TLS :8884
```

8883 belongs to the HAProxy public frontend on the **anchor IP** of ha-01/02. The Mosquitto processes listen on private backend port 8884, so the broker and public HAProxy listener never contend for the same address/port.

Why: HAProxy and Mosquitto cannot both bind the same `<HOST_PRIVATE_IP>:8883`. Using 8884 internally removes that collision while preserving the normal public MQTT endpoint `mqtt.<DOMAIN>:8883`.

The cloud brokers do not need a plaintext 1883 listener. The only plaintext 1883 in this architecture is the Raspberry Pi gateway-local loopback broker at `127.0.0.1:1883`.

8.2 Preconditions

Before installing either service, record:

```
ha-01 private IP:
ha-02 private IP:
ha-03 private IP:
DOMAIN:
confirmed ChirpStack region/topic prefix:
Mosquitto image/version/digest:
Valkey image/version/digest:
MQTT CA path:
Valkey CA path:
Valkey shared application secret reference:
Sentinel administration secret reference:
```

Required certificates already issued by the PKI step:

```
Mosquitto-1 server certificate
  SAN: mqtt.<DOMAIN>
  SAN: mqtt-ha.internal.<DOMAIN>
  SAN: ha-01 private name/IP as required

Mosquitto-2 server certificate
  SAN: mqtt.<DOMAIN>
  SAN: mqtt-ha.internal.<DOMAIN>
  SAN: ha-02 private name/IP as required

Valkey-1/2/3 server certificates
  SAN: valkey-ha.internal.<DOMAIN>
  SAN: each node's own private name/IP
```

MQTT client identities required for the POC:

```
one unique certificate per physical gateway
  CN = its 16-hex Gateway EUI

ChirpStack workload certificate(s)
  not reused by gateways

Node-RED workload certificate
  read-only application-uplink permission
```

Stop here. Do not continue if certificate SANs, private IPs, region prefix, or image versions are still placeholders.

8.3 Prepare Mosquitto on ha-01 and ha-02

Run this section on **ha-01 and ha-02 only**.

Create directories:

```
sudo install -d -m 750 /etc/lorawan-cloud/mosquitto
sudo install -d -m 750 /etc/lorawan-pki/mqtt
sudo install -d -m 750 /srv/mosquitto/data
sudo install -m 640 /dev/null /etc/lorawan-cloud/mosquitto/acl
```

Install the node's broker certificate/key and the gateway/workload CA using the owner/group required by the pinned Mosquitto image or package. Never copy a CA private key to a broker.

Create `/etc/lorawan-cloud/mosquitto/mosquitto.conf` on each broker host. Replace `<THIS_HOST_PRIVATE_IP>` with that host's VPC address:

```
persistence true
persistence_location /mosquitto/data/
autosave_interval 60
log_dest stdout

listener 8884 <THIS_HOST_PRIVATE_IP>
protocol mqtt
allow_anonymous false

cafile /mosquitto/config/certs/ca.crt
certfile /mosquitto/config/certs/server.crt
keyfile /mosquitto/config/certs/server.key
require_certificate true
use_identity_as_username true
tls_version tlsv1.2

acl_file /mosquitto/config/acl
```

`require_certificate true` makes the TLS listener require a valid client certificate. `use_identity_as_username true` maps the client-certificate Common Name to the MQTT username used by the ACL.

Do not add a cloud listener on 1883.

8.4 Create the MQTT ACL

For each gateway, add only its own topics. Example shape:

```
user <GATEWAY_EUI>
topic write <REGION_PREFIX>/gateway/<GATEWAY_EUI>/event/#
topic write <REGION_PREFIX>/gateway/<GATEWAY_EUI>/state/#
topic read <REGION_PREFIX>/gateway/<GATEWAY_EUI>/command/#
```

For ChirpStack, give the reviewed workload identity the gateway-backend permissions it actually needs:

```
user <CHIRPSTACK_MQTT_IDENTITY>
topic read <REGION_PREFIX>/gateway/+/event/#
topic read <REGION_PREFIX>/gateway/+/state/#
topic write <REGION_PREFIX>/gateway/+/command/#
topic write application/+/device/+/event/#
topic read application/+/device/+/command/#
```

For Node-RED:

```
user <NODE_RED_MQTT_IDENTITY>
topic read application/+/device/+/event/up
```

If the pinned ChirpStack configuration uses separate gateway-backend and application-integration certificates, split the ChirpStack ACL accordingly rather than broadening one identity.

Copy the **same reviewed ACL policy** to Mosquitto-1 and Mosquitto-2. Keep certificate/private-key files node-specific.

8.5 Run each Mosquitto backend

A containerized host-network example is below. Use the exact paths/user documented by the pinned image:

```
services:
  mosquitto:
    image: <PINNED_MOSQUITTO_IMAGE>
    network_mode: host
    restart: unless-stopped
    volumes:
      - /etc/lorawan-cloud/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - /etc/lorawan-cloud/mosquitto/acl:/mosquitto/config/acl:ro
      - /etc/lorawan-pki/mqtt:/mosquitto/config/certs:ro
      - /srv/mosquitto/data:/mosquitto/data
```

Validate before start using the validation method supported by the pinned image, then:

```
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml config --quiet
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml up -d
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml ps
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml logs --since=5m --tail=100
sudo ss -lntp | grep ':8884'
```

Pass only when the listener is on the intended private address and no cloud :1883 listener exists.

8.6 Prove Mosquitto TLS and ACL before HAProxy

From an approved VPC host, use a staging client certificate:

```
mosquitto_pub \
  -h <MOSQUITTO_PRIVATE_IP> -p 8884 \
  --cafile <MQTT_CA> \
  --cert <STAGING_CLIENT_CERT> \
  --key <STAGING_CLIENT_KEY> \
  -t '<AUTHORIZED_TEST_TOPIC>' \
  -m 'mqtt-ha-preflight' -q 1
```

Then attempt one topic that the identity is **not** allowed to publish/read. The authorized operation must work and the cross-identity operation must be denied.

A successful TLS handshake alone is not enough; ACL direction must also be proven.

8.7 Add the HAProxy MQTT frontends

Use the **same preferred/backup ordering on every HAProxy host** so clients do not split across two unrelated brokers.

On ha-01 and ha-02, add both frontends:

```
frontend mqtt_public
  mode tcp
  bind <THIS_HOST_ANCHOR_IP>:8883
  default_backend mosquitto_active_standby
```

```

frontend mqtt_internal
    mode tcp
    bind <THIS_HOST_PRIVATE_IP>:18883
    default_backend mosquitto_active_standby

backend mosquitto_active_standby
    mode tcp
    option tcp-check
    default-server inter 2s fall 3 rise 2
    server mosquitto-1 <HA01_PRIVATE_IP>:8884 check
    server mosquitto-2 <HA02_PRIVATE_IP>:8884 check backup

```

On ha-03, add **only** the private internal frontend plus the same backend:

```

frontend mqtt_internal
    mode tcp
    bind <HA03_PRIVATE_IP>:18883
    default_backend mosquitto_active_standby

backend mosquitto_active_standby
    mode tcp
    option tcp-check
    default-server inter 2s fall 3 rise 2
    server mosquitto-1 <HA01_PRIVATE_IP>:8884 check
    server mosquitto-2 <HA02_PRIVATE_IP>:8884 check backup

```

The runtime HAProxy check is intentionally a Layer-4 availability check. End-to-end certificate and ACL correctness is proven separately with actual MQTT clients because the broker requires client certificates.

Validate and reload on each affected host:

```

sudo haproxy -c -V -f /etc/haproxy/haproxy.cfg
sudo systemctl reload haproxy
sudo ss -lntp | grep -E ':(8883|18883)\b'

```

Expected:

```

ha-01: 8883 + 18883
ha-02: 8883 + 18883
ha-03: 18883 only

```

8.8 MQTT service naming: target design versus commissioned state

The original design below used a local-per-host service name on :18883. That route was **not** commissioned during Phase 8B and must not be treated as live:

```

mqtt-ha.internal.<DOMAIN>:18883    # target only; not commissioned

```

The currently validated cloud MQTT path is:

```

client TLS hostname/SNI: mqtt.internal.lorawan.com
HAProxy frontend:       10.104.0.2:8883
Mosquitto backends:     10.104.0.2:8884 and 10.104.0.4:8884
TLS termination:        Mosquitto, not HAProxy

```

HAProxy forwards MQTT TLS bytes unchanged, so clients must verify the certificate presented by the selected Mosquitto backend. Phase 9 must not start ChirpStack until it closes two remaining items: a ChirpStack-specific MQTT authentication/ACL identity, and a routing design that does not make the second ChirpStack instance depend on a single u1c-01 HAProxy process. If a local-per-app-node MQTT frontend is commissioned later, its

hostname must match an actually issued broker certificate identity; do not reuse the obsolete `mqtt-ha.internal.<DOMAIN>` placeholder unless new certificates explicitly contain it.

8.9 MQTT failover rehearsal

Before moving on to ChirpStack HA:

1. verify Mosquitto-1 is the selected backend from all three HAProxy hosts;
2. connect one staging MQTT client through `mqtt-ha.internal.<DOMAIN>:18883`;
3. publish/receive one authorized test message;
4. stop **Mosquitto-1 only**;
5. record the stop timestamp;
6. wait for HAProxy to mark it unavailable;
7. reconnect the staging client without changing hostname, certificate, or port;
8. prove Mosquitto-2 serves the connection;
9. restart Mosquitto-1;
10. verify it is healthy and returns to preferred status;
11. confirm no broker is unintentionally listening on cloud 1883.

Later, repeat the same failure using the physical gateway and its local QoS 1 queue.

Do not call this broker-state replication. Mosquitto-1 and Mosquitto-2 do not share sessions or queues.

8.10 Prepare Valkey storage and certificates

Run on **all three hosts**:

```
sudo install -d -m 750 /etc/lorawan-cloud/valkey
sudo install -d -m 750 /etc/lorawan-pki/valkey
sudo install -d -m 700 /srv/valkey/data
sudo install -d -m 750 /srv/valkey/sentinel
```

Use one protected long random application/replication secret for this tiny POC and a separate Sentinel-administration secret. Keep them outside Git and shell history.

8.11 Configure the initial Valkey primary and replicas

Use `ha-01` as the **bootstrap primary only**. Sentinel may later promote either replica.

Common secure baseline, adapted per host:

```
bind <THIS_HOST_PRIVATE_IP>
protected-mode yes
port 0
tls-port 6379

tls-cert-file /run/valkey-tls/server.crt
tls-key-file /run/valkey-tls/server.key
tls-ca-cert-file /run/valkey-tls/ca.crt
tls-auth-clients no
tls-replication yes

requirepass <VALKEY_SECRET>
masterauth <VALKEY_SECRET>
```

```
appendonly yes
appendfsync everysec
maxmemory 128mb
maxmemory-policy noeviction
```

On ha-02 and ha-03, additionally configure:

```
replicaof <HA01_PRIVATE_IP> 6379
```

Do not configure replicaof on the bootstrap primary ha-01.

noeviction is deliberate for the POC: memory exhaustion should fail visibly instead of silently discarding state.

If the pinned Valkey version uses ACLs instead of the password-only baseline, use a named least-privilege application/replication user and configure Sentinel authentication consistently. Do not mix incompatible ACL/password schemes accidentally.

8.12 Configure Sentinel on all three hosts

Create a writable sentinel.conf per host. Sentinel rewrites this file as topology changes, so do not mount it read-only.

Baseline:

```
bind <THIS_HOST_PRIVATE_IP>
protected-mode yes
port 0
tls-port 26379

tls-cert-file /run/valkey-tls/server.crt
tls-key-file /run/valkey-tls/server.key
tls-ca-cert-file /run/valkey-tls/ca.crt
tls-auth-clients no
tls-replication yes

requirepass <SENTINEL_SECRET>

sentinel monitor lorawan-valkey <HA01_PRIVATE_IP> 6379 2
sentinel auth-pass lorawan-valkey <VALKEY_SECRET>
sentinel down-after-milliseconds lorawan-valkey 5000
sentinel failover-timeout lorawan-valkey 60000
sentinel parallel-syncs lorawan-valkey 1
```

The 2 in sentinel monitor is the failure-detection quorum. Actual failover also needs a majority of the three Sentinel processes to authorize a leader for the failover.

Keep the Sentinel configuration/state path persistent and writable.

8.13 Run Valkey and Sentinel

Use the pinned Valkey image or package. If containerized, use stable host networking so Sentinel advertises the real VPC addresses instead of NAT-remapped container addresses.

A per-host container intent is:

```
services:
  valkey:
    image: <PINNED_VALKEY_IMAGE>
```

```

network_mode: host
restart: unless-stopped
command: ["valkey-server", "/etc/valkey/valkey.conf"]
volumes:
  - /etc/lorawan-cloud/valkey/valkey.conf:/etc/valkey/valkey.conf:ro
  - /etc/lorawan-pki/valkey:/run/valkey-tls:ro
  - /srv/valkey/data:/data

sentinel:
  image: <PINNED_VALKEY_IMAGE>
  network_mode: host
  restart: unless-stopped
  command: ["valkey-server", "/var/lib/valkey-sentinel/sentinel.conf", "--sentinel"]
  volumes:
    - /srv/valkey/sentinel:/var/lib/valkey-sentinel
    - /etc/lorawan-pki/valkey:/run/valkey-tls:ro

```

Place the host's writable `sentinel.conf` under `/srv/valkey/sentinel/` with ownership matching the image runtime user.

Validate the pinned image's binary/config paths before start.

8.14 Verify Valkey replication and Sentinel quorum

From an approved host with `valkey-cli`, use TLS and protected password input.

Check each data node:

```

valkey-cli --tls \
--cacert <VALKEY_CA> \
-h <VALKEY_PRIVATE_IP> -p 6379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE

```

Normal initial state:

```

ha-01 -> master/primary
ha-02 -> slave/replica
ha-03 -> slave/replica

```

Then query each Sentinel:

```

valkey-cli --tls \
--cacert <VALKEY_CA> \
-h <SENTINEL_PRIVATE_IP> -p 26379 \
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \
SENTINEL CKQUORUM lorawan-valkey

```

Pass only when all three Sentinels agree on the same primary and `CKQUORUM` reports enough Sentinels for quorum and failover authorization.

8.15 Validated HAProxy Valkey writable-primary routing

The live client-facing Valkey endpoints are:

```

ulc-01 -> 10.104.0.2:16379
ulc-02 -> 10.104.0.4:16379
TLS identity presented end-to-end -> valkey.internal.lorawan.com

```

Application traffic is TCP/TLS passthrough: HAProxy does **not** terminate or re-encrypt the client stream. HAProxy performs a separate TLS health connection to each Valkey backend, verifies the commissioned CA and SNI `valkey.internal.lorawan.com`, authenticates with the dedicated `haproxy-health` ACL identity loaded from a protected systemd EnvironmentFile, sends exact CRLF-framed `INFO replication`, and accepts only `role:master`.

The validated HAProxy 2.8 health-check shape is:

```
backend valkey_primary_backend
    mode tcp
    option tcp-check

    tcp-check connect ssl sni valkey.internal.lorawan.com
    tcp-check send-lf "AUTH haproxy-health %[env{VALKEY_HAPROXY_HEALTH_PASSWORD}]\r\n"
    tcp-check expect string +OK
    tcp-check send-lf "INFO replication\r\n"
    tcp-check expect min-recv 64 string role:master
```

The main Valkey application/replication password is **not** stored in `haproxy.cfg`. HAProxy receives only the separate least-privilege health credential. The `haproxy` service can read `/etc/haproxy/ca/valkey-ca.crt` but cannot read any Valkey node private key.

8.16 Validated Valkey endpoint and failover result

For a client on an application node, map `valkey.internal.lorawan.com` to that node's local HAProxy private IP and connect to `:16379`. A direct validation form is:

```
valkey-cli --tls \
--cacert <VALKEY_CA> \
--sni valkey.internal.lorawan.com \
-h <LOCAL_HAPROXY_PRIVATE_IP> -p 16379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' \
INFO replication
```

Expected routed role: `master`.

The controlled Phase 8C.14 failover proved the complete path: Sentinel promoted `ulc-02` after `ulc-03` Valkey was stopped; all three Sentinels agreed; both HAProxy `:16379` endpoints followed the new primary automatically without configuration changes or reloads; ten consecutive requests through each endpoint were master-only; pre-failover data survived; post-failover writes succeeded; `ulc-03` rejoined as a replica; and the final primary reported two connected replicas. At the Phase 8C.14 acceptance checkpoint, the Valkey primary was `ulc-02 (10.104.0.4)`. Treat Sentinel as authoritative for the live elected primary and do not manually fail back merely to match this dated checkpoint.

8.17 Final acceptance and Phase 9 handoff

Phase 8 core-service acceptance is split deliberately:

MQTT infrastructure accepted: Mosquitto 2.0.18 runs on `ulc-01/02`, both brokers serve TLS on `:8884`, `10.104.0.2:8883` provides validated HAProxy TLS passthrough, the shared broker certificate name `mqtt.internal.lorawan.com` verifies, HAProxy detects a failed broker, traffic can continue through the surviving broker, and service recovery has passed. This proves the broker/TLS/failover foundation only. ChirpStack workload authentication/ACLs, gateway identities, and a two-app-node MQTT route remain explicit later dependencies.

Valkey HA accepted: one primary and two replicas are maintained; three TLS Sentinels have quorum 2; failover-safe `masterauth` exists on every node; HAProxy exposes only the writable primary through both application-node

:16379 endpoints; automatic promotion, client routing convergence, data survival, writes after promotion, and old-primary replica recovery all passed.

Phase 9 may now begin with **read-only ChirpStack preflight and dependency closure**. Do not start ChirpStack until its exact image/config schema is pinned and the remaining MQTT workload identity/ACL plus second application-node routing boundary is closed.

8.18 Phase 8 Execution Record - MQTT Foundation Deployment

Recorded deployment evidence:

```
Date:
2026-08-24

MQTT baseline discovery:

ulc-01
10.104.0.2

ulc-02
10.104.0.4

ulc-03
10.104.0.8

Before installation:
- no MQTT packages installed
- no Valkey packages installed
- no MQTT listeners
- no Valkey listeners
- no MQTT/Valkey containers
- no MQTT/Valkey services

Mosquitto deployment:

Installed:
- mosquitto
- mosquitto-clients

Verified:
- Mosquitto version: 2.0.18
- MQTT protocol support: MQTT v5.0/v3.1.1/v3.1

Service state:

ulc-01:
mosquitto.service active (running)

ulc-02:
mosquitto.service active (running)

Result:
MQTT broker foundation installed successfully on the first two application nodes.
```

Next validation steps:

1. Configure Mosquitto TLS listeners.
2. Deploy MQTT CA and broker certificates.
3. Configure MQTT authentication and ACLs.
4. Validate broker-to-client TLS.
5. Add HAProxy MQTT routing.
6. Continue with Valkey deployment.

8.19 Phase 8 MQTT TLS Foundation Precheck Record

Executed on:

```
ulc-01  
10.104.0.2
```

```
ulc-02  
10.104.0.4
```

Discovery results:

```
Mosquitto:  
- installed  
- active  
- enabled  
  
MQTT PKI:  
- /etc/lorawan-pki/mqtt does not exist yet  
  
Existing PKI:  
- PostgreSQL TLS present  
- PgBouncer TLS present  
- MQTT TLS material not issued yet
```

Current listeners before TLS configuration:

```
ulc-01:  
127.0.0.1:1883  
[::1]:1883  
  
ulc-02:  
127.0.0.1:1883  
[::1]:1883
```

Interpretation:

- Mosquitto is running in the default local-only state.
- No cloud MQTT listener is exposed.
- No TLS listener exists yet.
- No plaintext remote MQTT exposure exists.

Next execution step:

1. Issue MQTT broker certificates from the project CA.
2. Install /etc/lorawan-pki/mqtt on broker nodes.
3. Replace local-only Mosquitto configuration with TLS listener configuration.
4. Validate mutual TLS before HAProxy integration.

Operational note:

Gateway client certificate provisioning is intentionally deferred until the final onboarding stage. Cloud MQTT broker TLS identities must be completed, HAProxy routing must be validated, Valkey and ChirpStack integration must be operational, and production cloud-side testing must pass before issuing gateway certificates.

Certificate separation:

- Broker certificates remain only on cloud MQTT nodes.
- Gateway certificates are separate client identities.

- Broker private keys are never transferred to gateways.

Automation preference:

Deployment steps should use chained command blocks where safe. Validation, backup, permission checks, and installation actions should be grouped into reproducible runbooks instead of many isolated commands.

Deployment ordering decision - Gateway certificate provisioning

Gateway MQTT client certificate provisioning is intentionally moved to the final deployment stage.

Reason:

- Cloud MQTT infrastructure must be completed and validated first.
- Mosquitto TLS, HAProxy routing, ACL policies, ChirpStack integration, and monitoring must exist before issuing gateway identities.
- Gateway certificates are production client credentials and should only be generated when the receiving MQTT service is ready.
- This prevents issuing gateway credentials against an unfinished broker architecture.

Final order:

1. Cloud foundation
 - etcd
 - Spilo/Patroni PostgreSQL HA
 - HAProxy
 - PgBouncer
2. Cloud application services
 - Mosquitto TLS
 - Valkey
 - ChirpStack
 - integrations
3. Cloud validation
 - TLS verification
 - failover testing
 - ACL testing
 - backup/recovery testing
4. Final gateway onboarding
 - issue gateway client certificate
 - install gateway CA/client certificate/key
 - configure MQTT Forwarder
 - validate end-to-end uplink

Gateway certificate provisioning must not begin before the cloud MQTT stack passes acceptance testing.

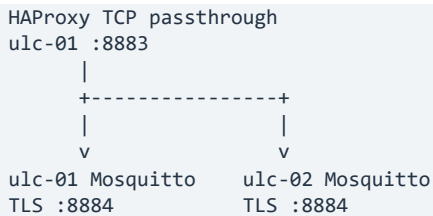
8.20 Phase 8B - MQTT HAProxy TLS Access Layer Execution Record

Objective:

Create a single MQTT TLS endpoint for gateways while keeping TLS termination on Mosquitto brokers.

Final design:

```
Gateway clients
|
| MQTT TLS :8883
|
```



Important design decision:

- HAProxy does not terminate MQTT TLS.
- Mosquitto retains broker certificates and private keys.
- HAProxy only forwards encrypted TCP traffic.
- Gateway certificates remain deferred until final onboarding.

8.20.1 HAProxy baseline check

Executed on ulc-01:

```
systemctl is-active haproxy
systemctl is-enabled haproxy
ss -H -lntp | grep haproxy
```

Existing services preserved:

```
PostgreSQL primary:
15432

PostgreSQL replicas:
15433
```

8.20.2 Initial HAProxy MQTT configuration attempt

The first configuration attempted:

```
frontend mqtt_tls
  bind 10.104.0.2:8883

backend mqtt_brokers
  server mqtt-ulc01 10.104.0.2:8883 check
  server mqtt-ulc02 10.104.0.4:8883 check
```

Validation passed:

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Runtime failed because Mosquitto already owned port 8883:

```
cannot bind socket
Address already in use
```

Correction:

- HAProxy owns the external MQTT port.
- Mosquitto moves to an internal TLS port.

8.20.3 Move Mosquitto internal TLS listener

Mosquitto configuration changed:

Before:

```
listener 8883
```

After:

```
listener 8884

cafile /etc/lorawan-pki/mqtt/ca.crt
certfile /etc/lorawan-pki/mqtt/server.crt
keyfile /etc/lorawan-pki/mqtt/server.key

tls_version tlsv1.3
require_certificate false
```

Restart:

```
systemctl restart mosquitto
```

Validation:

```
ss -H -lntp | grep mosquitto
```

Expected:

```
0.0.0.0:8884
[::]:8884
```

8.20.4 Final HAProxy MQTT configuration

Backup first:

```
cp /etc/haproxy/haproxy.cfg \
/etc/haproxy/haproxy.cfg.phase8b-before-$(date -u +%Y%m%d-%H%M%S)
```

Configuration added:

```
frontend mqtt_tls
    bind 10.104.0.2:8883
    mode tcp
    option tcplog
    default_backend mqtt_brokers

backend mqtt_brokers
    mode tcp
    option tcp-check

    # The two Mosquitto brokers do not replicate live MQTT sessions/topic state.
    # Keep all normal sessions on ulc-01 so a gateway's separate uplink/downlink
    # connections and ChirpStack MQTT sessions cannot be split across brokers.
    server mqtt-ulc01 10.104.0.2:8884 check
    server mqtt-ulc02 10.104.0.4:8884 check backup
```

Validate:

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Restart:

```
systemctl restart haproxy
```

Final listeners:

```
HAProxy:
10.104.0.2:8883

Mosquitto:
ulc-01 :8884
ulc-02 :8884
```

8.20.5 MQTT HA TLS validation

Client validation performed from ulc-03:

```
openssl s_client \
-connect 10.104.0.2:8883 \
-CAfile /etc/lorawan-pki/mqtt/ca.crt \
-verify_hostname mqtt.internal.lorawan.com \
-brief \
</dev/null
```

Result:

```
Protocol version: TLSv1.3
Cipher: TLS_AES_256_GCM_SHA384
Verification: OK
Verified peername: mqtt.internal.lorawan.com
```

PASS:

- HAProxy TLS passthrough
- broker certificate validation
- MQTT TLS endpoint

8.20.6 MQTT failover validation

Failure test:

Stop ulc-01 broker:

```
systemctl stop mosquitto
```

HAProxy detected failure:

```
Server mqtt_brokers/mqtt-ulc01 is DOWN
reason: Layer4 connection problem
```

Client test through HAProxy remained successful:

```
Verification: OK
```

Verified peername: mqtt.internal.lorawan.com

Recovery:

```
systemctl start mosquitto
```

HAProxy recovered the backend:

```
Server mqtt_brokers/mqtt-ulc01 is UP
reason: Layer4 check passed
2 active and 0 backup servers online
```

Phase 8B result:

MQTT HAProxy endpoint	PASS
TLS passthrough	PASS
ulc-01 Mosquitto backend	PASS
ulc-02 Mosquitto backend	PASS
Failover detection	PASS
Service recovery	PASS

8.21 Phase 8C - Valkey HA Deployment Execution Record

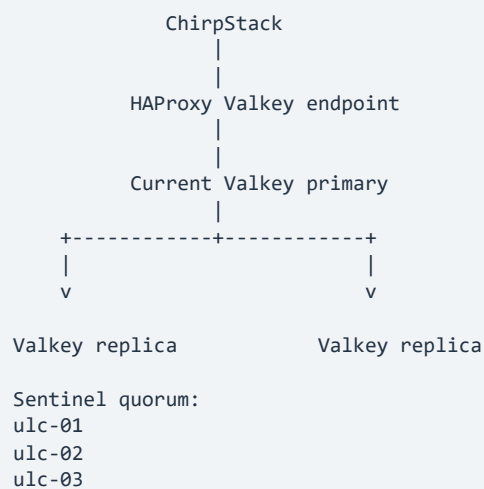
Objective:

Deploy the ChirpStack shared state backend using Valkey with future Sentinel-based high availability.

Execution boundary:

- Valkey deployment started after MQTT HA completion.
- Gateway provisioning remains deferred until cloud services are complete.
- No ChirpStack integration changes are performed until Valkey HA passes validation.
- No Git commit or push is performed during deployment sessions. Operator performs final review and commit.

Final intended architecture:



8.21.1 Valkey foundation precheck

Validated nodes:

```
ulc-01
10.104.0.2

ulc-02
10.104.0.4

ulc-03
10.104.0.8
```

Validation performed:

```
Redis/Valkey packages:
none installed

Redis/Valkey services:
none configured

Listeners:
6379 unavailable
26379 unavailable
```

Resource validation:

```
ulc-01:
RAM 1.9Gi
Available 1.3Gi

ulc-02:
RAM 1.9Gi
Available 1.4Gi

ulc-03:
RAM 1.9Gi
Available 1.4Gi
```

Result:

```
Phase 8C.1 Foundation Precheck: PASS
```

8.21.2 Valkey package installation

Deployment automation:

```
ssh \
-i /root/.ssh/cloud-deployment-phase8 \
-o IdentitiesOnly=yes \
opsadmin@NODE
```

The deployment key is required for multi-node automation. Do not rely on default SSH keys.

Installed package:

```
valkey-server
valkey-tools
```

Version installed:

Valkey 7.2.13

8.21.3 Installation status checkpoint

ulc-01

Status:

PASS

Verified:

```
valkey-server installed
service active
service enabled
```

Listener:

```
127.0.0.1:6379
[::1]:6379
```

ulc-02

Status:

PASS

Verified:

```
valkey-server installed
service active
service enabled
```

Listener:

```
127.0.0.1:6379
[::1]:6379
```

ulc-03

Status:

PASS

The first remote installation attempt failed because SSHing from ulc-03 back into opsadmin@10.104.0.8 did not provide a TTY for sudo. The corrected procedure installed Valkey locally on ulc-03 and then performed a local verification.

Verified final state:

```
Valkey version: 7.2.13
service: active
startup: enabled
```

```
listener: 127.0.0.1:6379 and [::1]:6379
PING: PONG
```

Final Phase 8C.2 foundation state:

ulc-01	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-02	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-03	Valkey 7.2.13	active	enabled	loopback:6379	PONG

Result:

```
Phase 8C.2 Valkey Foundation: PASS
```

No node exposes Valkey on the private VPC yet. Replication, authentication, TLS/private binding, Sentinel, and HAProxy routing remain intentionally deferred to Phase 8C.3 and later.

8.21.4 Next execution step

Resume from ulc-03. Do not SSH from ulc-03 back into itself for privileged package installation. The failed first attempt occurred because remote sudo on opsadmin@10.104.0.8 required a TTY. Install the remaining package locally with normal interactive sudo, then use the existing Phase 8 deployment key only to verify ulc-01 and ulc-02.

Use this single automation block:

```
sudo -v

sudo bash -euo pipefail <<'EOF'

echo '=== PHASE 8C.2 COMPLETE VALKEY FOUNDATION ==='

SSH_KEY='/root/.ssh/cloud-deployment-phase8'

echo
echo '=== STEP 1 - VERIFY THIS IS ULC-03 ==='

[ "$(hostname)" = 'ulc-03' ] || {
    echo 'FAIL: run this block on ulc-03 only'
    exit 1
}

echo 'host = ulc-03: PASS'

echo
echo '=== STEP 2 - INSTALL VALKEY LOCALLY ON ULC-03 ==='

export DEBIAN_FRONTEND=noninteractive
apt-get update
apt-get install -y valkey-server valkey-tools

systemctl enable valkey-server
systemctl start valkey-server

echo
echo '=== STEP 3 - VERIFY ULC-03 ==='

valkey-server --version
systemctl is-active valkey-server
systemctl is-enabled valkey-server
ss -H -lnt | grep -E '127\.0\.0\.1:6379|\[::1\]:6379' || {
    echo 'FAIL: ulc-03 local Valkey listener missing'
    exit 1
}

valkey-cli -h 127.0.0.1 -p 6379 PING | grep -Fxq PONG || {
    echo 'FAIL: ulc-03 Valkey PING failed'
    exit 1
}
```

```

}

echo 'ulc-03 Valkey foundation: PASS'

echo
echo '=== STEP 4 - VERIFY ALL THREE NODES ==='

for ENTRY in \
  'ulc-01|10.104.0.2' \
  'ulc-02|10.104.0.4'
do
  NAME="${ENTRY%|*}"
  IP="${ENTRY##*|}"

  echo
  echo "===== $NAME ($IP) ====="

  ssh \
    -i "$SSH_KEY" \
    -o IdentitiesOnly=yes \
    -o StrictHostKeyChecking=yes \
    "opsadmin@$IP" '
      set -e
      hostname
      valkey-server --version
      systemctl is-active valkey-server
      systemctl is-enabled valkey-server
      ss -H -lnt | grep -E "127\.0\.0\.1:6379|\[::1\]:6379"
      valkey-cli -h 127.0.0.1 -p 6379 PING
    '
done

echo
echo '===== ulc-03 (10.104.0.8) ====='
hostname
valkey-server --version
systemctl is-active valkey-server
systemctl is-enabled valkey-server
ss -H -lnt | grep -E "127\.0\.0\.1:6379|\[::1\]:6379"
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.2 VALKEY FOUNDATION: PASS ==='
echo 'All three nodes have Valkey installed, active, enabled, local-only, and responding to PING.'
echo 'STOP HERE. Do not expose 6379 or configure replication until Phase 8C.3 security configuration.'

EOF

```

The block performs four clear actions:

1. Confirms it is running on ulc-03.
2. Installs and starts Valkey locally on ulc-03, avoiding the self-SSH sudo problem.
3. Verifies version, service state, loopback-only listener, and PONG on ulc-03.
4. Verifies the same accepted foundation state on ulc-01 and ulc-02 through the retained deployment SSH key.

Expected final foundation state:

ulc-01	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-02	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-03	Valkey 7.2.13	active	enabled	loopback:6379	PONG

Latest verified execution evidence:

```

ulc-01 (10.104.0.2)
Server v=7.2.13
active
enabled

```

```

127.0.0.1:6379 LISTEN
[::1]:6379 LISTEN
PING -> PONG
PASS

ulc-02 (10.104.0.4)
Server v=7.2.13
active
enabled
127.0.0.1:6379 LISTEN
[::1]:6379 LISTEN
PING -> PONG
PASS

```

At this checkpoint, ulc-01 and ulc-02 are definitively accepted. Do not mark Phase 8C.2 complete until the Step 3/Step 5 output for ulc-03 also shows Valkey 7.2.13, active, enabled, loopback-only 6379, and PONG.

After all three pass, continue to Phase 8C.3 in this order:

1. Run the security/TLS preflight below on all three nodes.
2. Issue and deploy Valkey server certificates only after confirming the actual Ubuntu package paths, service account, and TLS capability.
3. Configure authentication plus private-network/TLS listeners.
4. Configure one primary and two replicas.
5. Deploy three Sentinel members with quorum 2.
6. Add HAProxy writable-primary routing.
7. Perform controlled Valkey failover and recovery testing.

8.21.5 Phase 8C.3 security/TLS preflight

Observed preflight checkpoint:

```

ulc-01: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf, loopback 6379, OpenSSL linked, PONG
ulc-02: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf, loopback 6379, OpenSSL linked, PONG
ulc-03: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf; output was interrupted by the systemctl pager
at "lines 1-4/4 (END)"

```

The ulc-03 interruption is not a Valkey failure. `systemctl show` opened the pager because this portion was executed locally in an interactive terminal. Press `q` to leave the pager, then run the no-pager continuation below. Do not mutate authentication, TLS listeners, replication, or Sentinel until the ulc-03 remainder is captured.

```

sudo bash -euo pipefail <<'EOF'

echo '=== PHASE 8C.3 ULC-03 PREFLIGHT CONTINUATION ==='

[ "$(hostname)" = 'ulc-03' ] || {
    echo 'FAIL: run on ulc-03 only'
    exit 1
}

echo
echo '--- systemd service identity ---'
SYSTEMD_PAGER=cat systemctl show valkey-server --no-pager \
    -p User \
    -p Group \
    -p FragmentPath \
    -p ExecStart

echo
echo '--- package configuration files ---'
dpkg -L valkey-server |
    grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true

```

```

echo
echo '--- configured security/persistence directives ---'
grep -En \
'^[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-file|tls-auth-
clients|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' \
/etc/valkey/valkey.conf || true

echo
echo '--- relevant defaults including commented directives ---'
grep -En \
'^[[:space:]]*#[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|tls-auth-clients|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' \
/etc/valkey/valkey.conf | head -80 || true

echo
echo '--- current listener ---'
ss -H -lntp | grep ':6379' || true

echo
echo '--- TLS capability evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null |
grep -Ei 'ssl|crypto' || true

echo
echo '--- local PING ---'
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.3 ULC-03 PREFLIGHT CONTINUATION COMPLETE ==='

EOF

```

Acceptance for this continuation is: service identity valkey:valkey, /etc/valkey/valkey.conf confirmed, loopback-only 6379, OpenSSL libraries linked, and PONG.

This step is read-only. It discovers the actual Valkey service account, configuration files, current bind/persistence settings, and TLS support before any security or replication mutation.

```

sudo -v
sudo bash -euo pipefail <<'EOF'
SSH_KEY='/root/.ssh/cloud-deployment-phase8'

echo '=== PHASE 8C.3 VALKEY SECURITY/TLS PREFLIGHT ==='

for ENTRY in 'ulc-01|10.104.0.2' 'ulc-02|10.104.0.4'; do
    NAME="${ENTRY%|*}"
    IP="${ENTRY##*|}"
    ssh -i "$SSH_KEY" -o IdentitiesOnly=yes -o StrictHostKeyChecking=yes "opsadmin@$IP" "bash -s -- '$NAME'"
<<'REMOTE'
set -euo pipefail
NAME="$1"
echo
echo "===== $NAME ====="
hostname
echo '--- package/version ---'
dpkg-query -W -f='${Status}|${Version}\n' valkey-server valkey-tools
valkey-server --version
echo '--- systemd ---'
systemctl show valkey-server -p User -p Group -p FragmentPath -p ExecStart
echo '--- config files ---'
dpkg -L valkey-server | grep -E '/etc/.*(valkey|redis).*\..conf$|/etc/default/' || true
echo '--- active directives ---'
grep -Eh '^[[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' /etc/valkey/*.conf /etc/valkey/*.conf.d/*
2>/dev/null || true
echo '--- listener ---'
ss -H -lntp | grep ':6379' || true
echo '--- TLS evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null | grep -Ei 'ssl|crypto' || true
valkey-server --help 2>&1 | grep -i tls || true
echo '--- ping ---'

```

```

valkey-cli -h 127.0.0.1 -p 6379 PING
REMOTE
done

echo
echo '==== ulc-03 ====='
hostname
echo '--- package/version ---'
dpkg-query -W -f='${Status}]${Version}\n' valkey-server valkey-tools
valkey-server --version
echo '--- systemd ---'
systemctl show valkey-server -p User -p Group -p FragmentPath -p ExecStart
echo '--- config files ---'
dpkg -L valkey-server | grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true
echo '--- active directives ---'
grep -Eh '^[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' /etc/valkey/*.conf /etc/valkey/*.conf.d/*
2>/dev/null || true
echo '--- listener ---'
ss -H -lntp | grep ':6379' || true
echo '--- TLS evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null | grep -Ei 'ssl|crypto' || true
valkey-server --help 2>&1 | grep -i tls || true
echo '--- ping ---'
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.3 PREFLIGHT COMPLETE ==='
EOF

```

Do not expose port 6379, set passwords, configure replication, or deploy Sentinel until this preflight output has been reviewed.

8.21.5 Phase 8C.3 security/TLS preflight result

The preflight passed on all three nodes.

Validated baseline:

```

Valkey server/tools: 7.2.13+dfsg1-0ubuntu0.1
Service user/group: valkey:valkey
Main config: /etc/valkey/valkey.conf
Systemd unit: /usr/lib/systemd/system/valkey-server.service
Current listener: loopback-only 6379
Local connectivity: PONG
TLS libraries: libssl.so.3 + libcrypto.so.3

```

The explicitly visible active baseline on ulc-03 is:

```

bind 127.0.0.1 -::1
protected-mode yes
port 6379
dbfilename dump.rdb
dir /var/lib/valkey
appendonly no

```

Result:

```

Phase 8C.3 security/TLS preflight: PASS

```

8.21.6 Phase 8C.4 Valkey TLS certificate issuance result

Certificate issuance completed successfully on ulc-03 from the existing internal CA.

Issuance directory:

```
/root/lorawan-pg-ca/valkey-issuance-20260825-003932
```

CA identity:

```
subject=CN = LoRaWAN PostgreSQL Internal CA
issuer=CN = LoRaWAN PostgreSQL Internal CA
SHA-256
fingerprint=99:00:4B:B3:2D:7D:78:FA:38:61:7C:78:89:6D:7A:7E:FF:9F:A6:10:FC:8F:07:D4:E2:5E:35:25:36:E6:CB:3E
```

Issued node identities:

```
ulc-01 -> valkey.internal.lorawan.com + ulc-01 + 10.104.0.2
ulc-02 -> valkey.internal.lorawan.com + ulc-02 + 10.104.0.4
ulc-03 -> valkey.internal.lorawan.com + ulc-03 + 10.104.0.8
```

Each certificate includes both `serverAuth` and `clientAuth` extended key usage so the same per-node identity can participate in TLS-protected Valkey replication and later Sentinel communication without sharing one private key across hosts.

Certificate-chain verification passed for all three nodes. Certificate/private-key public-key hashes also matched for all three nodes.

Recorded SHA-256 certificate fingerprints:

```
ulc-01 27:17:81:BA:E3:9A:8D:F8:AF:C0:CD:DA:1A:2D:DB:25:27:84:C4:65:B3:4D:B9:40:7A:32:89:F7:EB:AF:AC:0A
ulc-02 BD:DD:EF:C9:95:C7:CF:36:50:77:CE:42:38:97:B0:DB:77:CB:F3:2C:50:AD:41:98:9C:AA:D6:84:0A:43:17:A6
ulc-03 E5:5C:A8:BE:E1:C7:0D:9E:A8:5C:93:3B:4B:80:EF:B8:58:FD:47:FA:08:8C:5E:3C:6D:94:C4:4C:9F:B0:4B:DC
```

Result:

```
Phase 8C.4 Valkey TLS certificate issuance: PASS
```

Keep the complete issuance directory until the Valkey/Sentinel HA layer and final security cleanup are complete. Do not delete the deployment SSH key or node private keys during commissioning.

8.21.7 Phase 8C.5 Valkey TLS material installation checkpoint

The remote TLS material installation completed successfully on `ulc-01` and `ulc-02`.

Validated on both nodes:

```
/etc/lorawan-pki/valkey          750 root:valkey
/etc/lorawan-pki/valkey/ca.crt    640 root:valkey
/etc/lorawan-pki/valkey/server.crt 640 root:valkey
/etc/lorawan-pki/valkey/server.key 640 root:valkey

ca-read-PASS
cert-read-PASS
key-read-PASS
server.crt: OK
```

Observed file sizes on `ulc-01` and `ulc-02`:

```
ca.crt      1891 bytes
server.crt  1627 bytes
```

```
server.key 1704 bytes
```

Final Phase 8C.5 status:

```
ulc-01 TLS material: PASS
ulc-02 TLS material: PASS
ulc-03 TLS material: PASS
```

ulc-03 local verification confirmed:

```
/etc/lorawan-pki/valkey          750 root:valkey
/etc/lorawan-pki/valkey/ca.crt    640 root:valkey (1891 bytes)
/etc/lorawan-pki/valkey/server.crt 640 root:valkey (1627 bytes)
/etc/lorawan-pki/valkey/server.key 640 root:valkey (1708 bytes)
ca-read-PASS
cert-read-PASS
key-read-PASS
/etc/lorawan-pki/valkey/server.crt: OK
certificate/private-key hash match: PASS
```

The Valkey service was deliberately left unchanged after PKI staging. ulc-03 remained active, listening only on loopback plaintext 6379, and returned PONG. This proves certificate installation itself did not alter runtime behavior.

Result:

```
Phase 8C.5 Valkey TLS material installation: PASS
```

8.21.8 Phase 8C.6 Valkey TLS + authentication result

Phase 8C.6 passed on all three nodes.

Validated runtime state:

```
ulc-01 10.104.0.2:6379 TLS-only + password auth: PASS
ulc-02 10.104.0.4:6379 TLS-only + password auth: PASS
ulc-03 10.104.0.8:6379 TLS-only + password auth: PASS
```

Security checks passed on every node:

```
service active + enabled
TLSv1.3 negotiated
cipher TLS_AES_256_GCM_SHA384
CA verification OK
peer name valkey.internal.lorawan.com verified
plaintext connection rejected
TLS without password rejected with NOAUTH
TLS with password returned PONG
tls-replication yes prepared
```

The shared Valkey authentication secret is retained only in the root-owned secret file created during commissioning and was not printed in the execution record.

Result:

```
Phase 8C.6 Valkey TLS + authentication: PASS
```

8.21.9 Phase 8C.7 initial replication attempt and authentication correction

The first Phase 8C.7 replication block stopped before any replica configuration was changed. Its primary precheck attempted to authenticate `valkey-cli` using the `VALKEYCLI_AUTH` environment variable, but the observed Ubuntu package behavior returned:

```
exit_code=0
NOAUTH Authentication required.
```

This is a command-client authentication issue, not a Valkey TLS or server failure. Phase 8C.6 already proved authenticated TLS PONG using the explicit `-a` option. The replication attempt stopped at Step 1, so `ulc-01/02/03` remain in the Phase 8C.6 standalone TLS-only authenticated state and no `replicaof` directive has yet been applied.

Correction: all remaining commissioning commands use explicit `-a "$AUTH"` for `valkey-cli` rather than relying on the environment-variable path. The secret remains unprinted and root-only. Valkey's upstream CLI documentation supports both `-a` and `VALKEYCLI_AUTH`, but actual observed package behavior is authoritative for this deployment.

The corrected Phase 8C.7 replication deployment then passed.

Final topology:

```
ulc-01 10.104.0.2 = PRIMARY
ulc-02 10.104.0.4 = REPLICa
ulc-03 10.104.0.8 = REPLICa
```

Observed primary state:

```
role:master
connected_slaves:2
slave0:ip=10.104.0.4,port=6379,state=online,offset=14,lag=0
slave1:ip=10.104.0.8,port=6379,state=online,offset=14,lag=0
```

Both replicas reported `role:slave`, `master_host:10.104.0.2`, `master_port:6379`, and `master_link_status:up`. Primary write propagation passed to both replicas, and both replicas correctly rejected direct writes with `READONLY`.

Result:

```
Phase 8C.7 Valkey TLS replication: PASS
```

8.21.10 Phase 8C.8 Sentinel foundation precheck checkpoint

The Sentinel foundation precheck has started. `ulc-01` completed successfully and showed:

```
no separate Sentinel package installed
valkey-sentinel package candidate: 7.2.13+dfsg1-0ubuntu0.1
redis-sentinel package candidate also exists but is not selected
/usr/bin/valkey-server present
no Sentinel systemd unit currently installed
valkey-server supports Sentinel mode via --sentinel
port 26379 free
```

This confirms the correct family is the matching `valkey-sentinel 7.2.13` package, not the older Redis Sentinel package.

The first continuation attempt reached `ulc-02` and confirmed:

```
valkey-sentinel candidate: 7.2.13+dfsg1-0ubuntu0.1
valkey-sentinel not installed
/usr/bin/valkey-server present
valkey-server supports Sentinel mode via --sentinel
```

A second continuation completed the remaining ulc-02 checks:

```
no Sentinel systemd unit
26379 free
```

ulc-02 therefore matches the validated ulc-01 Sentinel foundation baseline: matching valkey-sentinel 7.2.13 candidate, package not installed, Sentinel mode supported by valkey-server, no Sentinel unit yet, and port 26379 free.

The ulc-03 precheck then confirmed the host, matching valkey-sentinel candidate 7.2.13+dfsg1-0ubuntu0.1, package-not-installed state, /usr/bin/valkey-server, and visible Sentinel-mode support. The script stopped immediately after printing the Sentinel-mode help text, before the explicit PASS line, systemd-unit check, and port 26379 check.

The next ulc-03 continuation also stopped at the Sentinel capability step before any package/config/service mutation. The command used `HELP="$(valkey-server --help 2>&1)"` while the wrapper was running with `set -e`. On this Ubuntu Valkey build, valkey-server --help prints valid help text but exits non-zero, so the command substitution assignment itself triggers `set -e` and terminates the script before the captured text can be checked. The visible earlier output already proves that the help text contains `Sentinel mode;`; this is a shell-control-flow issue, not a Sentinel capability failure.

The corrected ulc-03 continuation passed. Final Phase 8C.8 Sentinel foundation precheck state:

```
ulc-01: PASS
ulc-02: PASS
ulc-03: PASS
valkey-sentinel candidate = 7.2.13+dfsg1-0ubuntu0.1
Sentinel package not installed
valkey-server supports --sentinel
no Sentinel systemd unit
26379 free
```

Result:

```
Phase 8C.8 Sentinel foundation precheck: PASS
```

Phase 8C.9 installed and inspected the matching valkey-sentinel package on all three nodes. Observed package version on ulc-01, ulc-02, and ulc-03: 7.2.13+dfsg1-0ubuntu0.1.

Package/runtime evidence:

```
executable: /usr/bin/valkey-sentinel
service: /usr/lib/systemd/system/valkey-sentinel.service
ExecStart: /usr/bin/valkey-sentinel /etc/valkey/sentinel.conf --supervised systemd --daemonize no
User: valkey
Group: valkey
configuration: /etc/valkey/sentinel.conf
configuration permissions: 640 valkey:valkey
RuntimeDirectory: sentinel
ReadWriteDirectories: /etc/valkey
```

The packaged default configuration is not suitable for production as-is:

```
protected-mode no
port 26379
sentinel monitor mymaster 127.0.0.1 6379 2
```

Immediately after package installation, valkey-sentinel was stopped and disabled on every node. Final safety verification passed on all three nodes:

```
ActiveState=inactive
UnitFileState=disabled
26379 closed: PASS
```

Result:

```
Phase 8C.9 Sentinel package foundation: PASS
```

No Sentinel quorum is active yet. Do not start or enable the service with the packaged default configuration.

Phase 8C.10 Sentinel TLS configuration preflight passed on ulc-03. Sentinel remained inactive and disabled, and port 26379 remained closed. The packaged Sentinel configuration exposes only the default plaintext port 26379 directive and the default sentinel monitor mymaster 127.0.0.1 6379 2; no TLS directives are pre-populated in /etc/valkey/sentinel.conf.

The valkey service account can read all staged TLS files (ca.crt, server.crt, and server.key, each 640 root:valkey). The current Valkey topology was also reconfirmed before Sentinel configuration:

```
role:master
connected_slaves:2
```

The existing writable primary remains 10.104.0.2:6379 with both replicas connected.

Valkey's TLS documentation confirms that Sentinel inherits the common TLS directives and that tls-replication yes controls both Sentinel-to-Valkey TLS and TLS on Sentinel's own inter-Sentinel listener. Therefore the production Sentinel configuration will explicitly disable plaintext with port 0, enable tls-port 26379, set the staged certificate/key/CA paths, set tls-replication yes, and retain quorum 2.

Result:

```
Phase 8C.10 Sentinel TLS configuration preflight: PASS
```

Phase 8C.11 TLS-only Sentinel quorum deployment progressed successfully through quorum validation. All three Sentinels started with node-specific private binds, plaintext disabled (port 0), TLS-only tls-port 26379, the staged certificate/key/CA files, tls-auth-clients no, tls-replication yes, separate Sentinel password authentication, and monitoring of lorawan-valkey at the current primary 10.104.0.2:6379 with quorum 2.

Validated runtime evidence:

```
ulc-01 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.2:26379
ulc-02 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.4:26379
ulc-03 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.8:26379
TLSv1.3 handshake to all three: PASS
CA verification: OK
peer name valkey.internal.lorawan.com: verified
all three Sentinels report primary 10.104.0.2:6379
two Sentinel peers discovered: PASS
CKQUORUM: OK 3 usable Sentinels. Quorum and failover authorization can be reached
```

The enablement continuation then confirmed ulc-03 was successfully enabled and remained active. Final service-state verification also passed on ulc-01 and ulc-02:

```
ulc-03 service=active, enabled=enabled
ulc-01 service=active, enabled=enabled, listeners 127.0.0.1:26379 + 10.104.0.2:26379
ulc-02 service=active, enabled=enabled, listeners 127.0.0.1:26379 + 10.104.0.4:26379
```

The final Sentinel verification continuation passed completely. ulc-03 exposes the expected TLS-only listeners on 127.0.0.1:26379 and 10.104.0.8:26379; all three Sentinel members report the same current primary 10.104.0.2:6379; ulc-03 sees exactly two Sentinel peers; and SENTINEL CKQUORUM lorawan-valkey returns OK 3 usable Sentinels. Quorum and failover authorization can be reached.

Final Phase 8C.11 state:

```
ulc-01 Sentinel = active + enabled
ulc-02 Sentinel = active + enabled
ulc-03 Sentinel = active + enabled
Sentinel transport = TLS-only :26379
Sentinel members = 3
quorum = 2
current primary = 10.104.0.2:6379
peer_count = 2
primary discovery = PASS on all three members
CKQUORUM = PASS
```

Result:

```
Phase 8C.11 TLS Sentinel quorum: PASS
```

Phase 8C.12 controlled automatic failover testing started. The starting-state check passed with Sentinel reporting 10.104.0.2:6379 as the primary, CKQUORUM returned OK 3 usable Sentinels, and the pre-failover test write to ulc-01 succeeded.

The script then stopped immediately at Step 4 while attempting to run `sudo systemctl stop valkey-server` remotely on ulc-01 over SSH. No output confirmed that the service actually stopped, and no automatic promotion output followed. Therefore automatic failover has **not yet been proven** and the test must continue from a verified current-state check rather than assuming ulc-01 went down.

This matches the earlier commissioning pattern where remote `sudo` from ulc-03 can terminate a non-interactive SSH block when `sudo` requires a terminal/password path. The correction is to first inspect ulc-01's actual Valkey service state and Sentinel's current master view. If ulc-01 is still active, stop it with an SSH allocation that permits the remote `sudo` interaction, while leaving `valkey-sentinel` running. If it is already inactive, do not stop it again. Then continue observing Sentinel without manually changing replication.

The corrected Phase 8C.12 continuation immediately observed that Sentinel's current primary had already changed to 10.104.0.8 (ulc-03):

```
=== PHASE 8C.12 AUTOMATIC FAILOVER CONTINUATION ===
=== STEP 1 - DISCOVER CURRENT STATE ===
Sentinel current primary = 10.104.0.8
```

This is strong evidence that the previous failure injection actually succeeded before the shell returned: ulc-01 stopped long enough for Sentinel to complete an automatic election and promote ulc-03. Do not stop any Valkey node again and do not manually issue `replicaof`. The remaining work is recovery verification, not another failover trigger.

Phase 8C.12 recovery verification then advanced substantially. All three Sentinels agreed on ulc-03 (10.104.0.8) as the elected primary. ulc-03 reported `role:master` with ulc-02 already online as a replica. The original pre-failover test key `phase8c12:failover:1787621601` was still present on ulc-03 with its expected `before-*` value, proving the pre-failover write survived the automatic promotion. A fresh authenticated TLS write to ulc-03 also succeeded.

Observed checkpoint:

```
ulc-01 Sentinel reports primary=10.104.0.8
ulc-02 Sentinel reports primary=10.104.0.8
ulc-03 Sentinel reports primary=10.104.0.8
three-Sentinel consensus: PASS
ulc-03 role:master
connected_slaves:1
ulc-02 online as replica
pre-failover data survived: PASS
new-primary write: PASS
```

The pasted output then ended at STEP 6 - INSPECT OLD PRIMARY ULC-01 before the ulc-01 service-state result was printed. A later minimized recovery block also stopped immediately at STEP 1 - CHECK ULC-01 SSH + VALKEY STATE with no remote output. This means the remaining problem is specifically the ulc-03 -> ulc-01 SSH inspection path, not the already-proven Sentinel failover. Do not trigger another failover and do not modify replication manually.

Phase 8C.12 is now proven through automatic promotion, Sentinel consensus, data survival, and successful writes on the promoted primary.

The follow-up SSH diagnostic to ulc-01 also completed successfully. SSH authentication with `/root/.ssh/cloud-deployment-phase8` is healthy and the remote host key matches. The remote service state is:

```
host=ulc-01
valkey=inactive
sentinel=active
ssh exit status=0
```

This confirms the original failure injection succeeded exactly as intended: only `valkey-server` on ulc-01 is down, while the local Sentinel stayed alive and participated in the election. There is no SSH or host reachability problem.

The recovery step then successfully started `valkey-server` on ulc-01. Immediate service verification showed:

```
valkey=active
sentinel=active
```

No replication directive was changed manually and ulc-03 remains the Sentinel-elected primary.

The final read-only recovery verification is currently waiting for ulc-01 to be reconfigured by Sentinel after its service restart. The observed output has reached at least attempt 18 of the bounded 60-attempt loop without yet matching all required replica fields. This is not yet a failure because the script intentionally allows up to 120 seconds (60 attempts at 2-second intervals) for recovery convergence.

The bounded rejoin wait did not immediately converge, so a read-only Sentinel diagnostic was captured. Sentinel still reports ulc-03 (10.104.0.8:6379) as the healthy master, with `num-slaves=2`, `num-other-sentinels=2`, quorum 2, and failover epoch 1.

Sentinel's replica view now proves that ulc-01 has been logically converted to a replica of ulc-03:

```
ulc-02 10.104.0.4:6379
flags=slave
master-host=10.104.0.8
master-port=6379
master-link-status=ok
```

```
ulc-01 10.104.0.2:6379
flags=slave
master-host=10.104.0.8
master-port=6379
master-link-status=err
```

The ulc-01 Sentinel log confirms the automatic recovery action itself occurred:

```
-sdown slave 10.104.0.2:6379 ... @ lorawan-valkey 10.104.0.8 6379
+convert-to-slave slave 10.104.0.2:6379 ... @ lorawan-valkey 10.104.0.8 6379
```

Therefore Sentinel failover and automatic topology rewriting are proven. The only remaining recovery defect is the data-plane replication link from ulc-01 to the elected primary: Sentinel sees ulc-01 as a slave of 10.104.0.8, but reports master-link-status=err.

The first read-only replication-link diagnostic captured the following ulc-01 state:

```
role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:down
master_last_io_seconds_ago:-1
master_sync_in_progress:1
master_sync_total_bytes:-1
master_sync_read_bytes:0
master_sync_left_bytes:-1
connected_slaves:0
```

This proves ulc-01 has accepted the Sentinel-directed replica role and is attempting an initial/full synchronization, but the data connection has not progressed far enough to receive any bytes. The diagnostic then stopped at the configuration-inspection step because opsadmin cannot read /etc/valkey/valkey.conf directly:

```
grep: /etc/valkey/valkey.conf: Permission denied
```

That permission error is from the diagnostic command only; it is not evidence of a Valkey runtime failure. The configuration file is intentionally restricted.

The corrected privileged config read then exposed an important clue on ulc-01:

```
replica-read-only yes
bind 127.0.0.1 10.104.0.2
port 0
tls-port 6379
tls-cert-file /etc/lorawan-pki/valkey/server.crt
tls-key-file /etc/lorawan-pki/valkey/server.key
tls-ca-cert-file /etc/lorawan-pki/valkey/ca.crt
tls-auth-clients no
tls-replication yes
replicaof 10.104.0.8 6379
```

masterauth is notably absent from the active replication/security directives even though the Valkey primary requires password authentication. That is a strong candidate explanation for the stalled full sync: Sentinel correctly rewrote replicaof to the newly elected primary, but the restarted former primary may no longer possess the credential needed for replica-to-primary authentication.

The direct TLS test in the same block failed only because it was run as opsadmin, which cannot read /etc/lorawan-pki/valkey/ca.crt (root:valkey 640). TCP reachability to 10.104.0.8:6379 passed, so that CA permission error was a diagnostic-user artifact, not proof of a TLS transport failure.

The follow-up confirmation then proved the transport and primary credentials are healthy:

```
masterauth/masteruser absent from /etc/valkey/valkey.conf
TLSv1.3 handshake from ulc-01 to 10.104.0.8:6379: PASS
CA verification: OK
Verified peername: valkey.internal.lorawan.com
authenticated valkey-cli PING from ulc-01 to ulc-03: PONG
ulc-01 role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:down
```

The attempted runtime `CONFIG GET masterauth` returned `NOAUTH Authentication required` because that local diagnostic command itself did not authenticate, so it does not establish the runtime value. However, the persistent configuration definitely lacks `masterauth`, while a manually authenticated TLS client from the same host reaches the elected primary successfully. Combined with the stalled replica link, this isolates the commissioning defect to the missing replica authentication credential rather than network, CA, hostname verification, or primary availability.

Root cause: Phase 8C.7 placed `masterauth` only on the nodes that were replicas at that moment. `ulc-01` was the original primary and therefore did not receive a persistent `masterauth`. After automatic failover, Sentinel correctly changed `ulc-01` to follow `ulc-03`, but Sentinel's topology rewrite does not supply the persistent Valkey replication password. For failover-safe symmetry, every Valkey node must retain the same `masterauth` even while it is primary; the directive is inert while a node is primary and becomes necessary if that node later becomes a replica.

The `masterauth` repair was then applied successfully on `ulc-01`. A timestamped backup was created at `/etc/valkey/valkey.conf.phase8c12-masterauth-before-20260825-020750`; the Sentinel-selected `replicaof 10.104.0.8 6379` directive was explicitly verified and preserved; exactly one persistent `masterauth` directive was written; `CONFIG SET masterauth` returned `OK`; and an authenticated `CONFIG GET masterauth` matched the expected secret without printing it.

Observed repair evidence:

```
replicaof 10.104.0.8 6379
masterauth lines=1
persistent masterauth: PASS
runtime masterauth: PASS
runtime masterauth verification: PASS
ulc-01 masterauth repair complete
```

The combined automation then dropped into an interactive root shell on `ulc-01` instead of returning cleanly to the outer `ulc-03` script, so the remaining verification was rerun as a separate continuation from `ulc-03`. This was an orchestration/TTY here-document issue, not a repair failure.

The post-repair verification immediately confirmed that the missing `masterauth` was the actual recovery defect. On the first check after applying the credential, `ulc-01` reported:

```
attempt=1 link=up sync=0
role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:up
master_last_io_seconds_ago:1
master_sync_in_progress:0
ulc-01 replication recovery: PASS
ulc-01 persistent masterauth: PASS
```

This is direct causal evidence: no `replicaof` change, service restart, or fallback was performed; adding the correct persistent/runtime `masterauth` alone restored the replica link to the Sentinel-elected primary.

The next all-node symmetry audit then failed on `ulc-02`:

```
FAIL: ulc-02 masterauth mismatch
```

Therefore Phase 8C.12 recovery is not yet fully closed. ulc-01 is healthy again, but failover-safe authentication is not normalized across all three Valkey nodes. The correct design is for every node to retain the same masterauth even when currently primary, because any node can later be converted into a replica by Sentinel.

The final failover-safe masterauth normalization then completed successfully on ulc-02 and ulc-03 without restarting Valkey and without altering Sentinel-selected topology. ulc-02 preserved replicaof 10.104.0.8 6379, received exactly one persistent masterauth, accepted the same value at runtime, and passed secret-equality verification without printing the credential. ulc-03 remained role:master, received the same persistent/runtime masterauth, and therefore all three nodes now carry the credential required if Sentinel later converts any current or former primary into a replica.

Final recovered topology:

```
ulc-03 10.104.0.8 = PRIMARY
ulc-01 10.104.0.2 = REPLICAS -> 10.104.0.8:6379, master_link_status:up
ulc-02 10.104.0.4 = REPLICAS -> 10.104.0.8:6379, master_link_status:up
ulc-03 connected_slaves:2
```

Final replication evidence:

```
slave0:ip=10.104.0.4,port=6379,state=online,offset=623818,lag=0
slave1:ip=10.104.0.2,port=6379,state=online,offset=623536,lag=0
primary write: PASS
ulc-01 write propagation: PASS
ulc-02 write propagation: PASS
```

All three Sentinels continued to report 10.104.0.8 as the primary, and SENTINEL CKQUORUM lorawan-valkey returned:

```
OK 3 usable Sentinels. Quorum and failover authorization can be reached
```

Phase 8C.12 is therefore fully closed:

```
Automatic Sentinel promotion = PASS
Pre-failover data survival = PASS
New-primary writes = PASS
ulc-01 masterauth repair = PASS
ulc-01 replication recovery = PASS
ulc-02 masterauth normalization = PASS
ulc-03 masterauth normalization = PASS
Failover-safe masterauth on every node = PASS
One primary + two replicas = PASS
Final write propagation = PASS
Three-Sentinel consensus = PASS
Quorum after recovery = PASS
```

Result:

```
Phase 8C.12 automatic failover + recovery: PASS
```

Keep ulc-03 as the Sentinel-elected primary. Do not manually fail back to ulc-01.

Next: Phase 8C.13 HAProxy writable-primary routing. Before mutation, preflight the existing HAProxy 2.8 deployment on the application nodes, confirm port 16379 is free, confirm the HAProxy service identity and CA-read path, and validate the exact TLS health-check syntax against the installed HAProxy build. The Valkey node

certificates contain the shared SAN `valkey.internal.lorawan.com`; do not use an unissued `valkey-ha.internal.lorawan.com` name for backend certificate verification.

The first Phase 8C.13 read-only preflight reconfirmed the Valkey/Sentinel baseline before inspecting HAProxy:

```
ulc-03 role:master
connected_slaves:2
ulc-01 + ulc-02 replicas online
Sentinel primary: 10.104.0.8:6379
Valkey topology: PASS
Sentinel primary: PASS
```

The script then reached `ulc-01` successfully but stopped at the HAProxy package-version command with:

```
bash: line 12: Status: unbound variable
```

Cause: the remote shell was running `set -u` and the `dpkg-query -f='${Status}|${Version}\n'` format string used double quotes. Bash attempted to expand `${Status}` and `${Version}` as shell variables before `dpkg-query` could interpret them, and the unset `Status` variable terminated the preflight. No HAProxy configuration, listener, or service state was changed.

Correction: quote the `dpkg-query` format with single quotes (`-f='${Status}|${Version}\n'`) inside the remote shell. Resume the Phase 8C.13 preflight from the HAProxy-node inspection; there is no need to repeat the already-passed Valkey/Sentinel baseline unless topology changes before the retry.

The corrected continuation then advanced through `ulc-01` and exposed the next concrete preflight issue. HAProxy is the expected pinned build `2.8.16-0ubuntu0.24.04.3`, active and enabled, with existing PostgreSQL listeners `10.104.0.2:15432/15433`, MQTT frontend `10.104.0.2:8883`, and Mosquitto backend listener `8884`; target port `16379` is free. `/etc/haproxy/haproxy.cfg` is `0644 root:root` and the expected existing PostgreSQL/MQTT sections remain present.

The Valkey CA cannot be traversed by the HAProxy service account because `/etc/lorawan-pki/valkey` is `0750 root:valkey`; `namei` therefore ends with `ca.crt - Permission denied for non-valkey users`. Do **not** add `haproxy` to the `valkey` group, because that group can also read the Valkey server private key (`0640 root:valkey`). Instead, install a CA-only copy for HAProxy under a dedicated HAProxy-readable path such as `/etc/haproxy/ca/valkey-ca.crt`, with no server certificate or private key copied. Validate certificate equality by SHA-256 before use.

The CA-only HAProxy trust-path correction then completed successfully on both application nodes. The commissioned Valkey CA hash on `ulc-03` is

`6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`, and the copied CA on both `ulc-01` and `ulc-02` matched this SHA-256 exactly.

Installed trust boundary on both HAProxy nodes:

```
/etc/haproxy/ca                750 root:haproxy
/etc/haproxy/ca/valkey-ca.crt   640 root:haproxy
HAProxy CA read: PASS
Valkey private key remains inaccessible to HAProxy: PASS
CA equality: PASS
```

The original `/etc/lorawan-pki/valkey` permissions were not weakened and the HAProxy account was not added to the `valkey` group. This preserves separation between CA trust and private-key access.

The remaining HAProxy preflight evidence passed on both `ulc-01` and `ulc-02`:

```
haproxy 2.8.16-0ubuntu0.24.04.3
service active + enabled
16379 free
```

```
existing PostgreSQL/MQTT listeners preserved
current /etc/haproxy/haproxy.cfg syntax valid
HAProxy dedicated CA readable
10.104.0.2:6379 reachable
10.104.0.4:6379 reachable
10.104.0.8:6379 reachable
```

The pasted execution ended after the ulc-02 TCP-backend reachability checks, before the separate all-backend TLS hostname-verification section printed. Therefore the CA trust staging and TCP preflight are accepted, but Phase 8C.13 is not yet closed.

The TLS identity and current-role portion of the next Phase 8C.13 preflight passed completely. All three Valkey backends negotiated TLS 1.3 with TLS_AES_256_GCM_SHA384, verified against the commissioned CA, and matched the shared certificate name `valkey.internal.lorawan.com`. Runtime role checks reconfirmed ulc-01 and ulc-02 as replicas following 10.104.0.8:6379 with links up, and ulc-03 as `role:master` with two connected replicas.

The first offline HAProxy TLS-backend syntax test failed before any production mutation. The temporary test config split one server directive across multiple physical lines without continuation syntax:

```
server test-primary
  10.104.0.8:6379
  check
  ssl
  verify required
  ca-file /etc/haproxy/ca/valkey-ca.crt
  sni str(valkey.internal.lorawan.com)
```

HAProxy 2.8 therefore parsed each subsequent line as a separate backend keyword and reported unknown keyword `'10.104.0.8:6379'`, followed by equivalent errors for `check`, `ssl`, `verify`, `ca-file`, and `sni`. This is a configuration-formatting error, not rejection of TLS server parameters. No production HAProxy config was changed or reloaded.

Correction: keep the full server directive on one logical line (or use explicit shell-generated single-line output), then validate the TLS backend syntax offline. After that, validate the exact `tcp-check connect/send/expect` syntax separately before introducing any health-check credential.

The corrected single-line server syntax was accepted by HAProxy, but the temporary test file contained only a backend and no frontend/listen section. `haproxy -c` therefore printed:

```
Configuration file has no error but will not start (no listener) => exit(2).
```

This is not a syntax failure. It explicitly confirms that HAProxy found no configuration error; exit code 2 is produced because the isolated test file has no listener and therefore would not start as a runtime configuration. No production config was changed or reloaded.

The corrected offline validation then passed on both ulc-01 and ulc-02 by adding a harmless dummy loopback frontend only inside the temporary test configuration. No listener was actually started because validation still used `haproxy -c` only.

Validated on both HAProxy nodes:

```
TLS backend server-line syntax: PASS
tcp-check TLS connect syntax: PASS
tcp-check send/expect syntax: PASS
combined three-backend TLS health-check grammar: PASS
production /etc/haproxy/haproxy.cfg remains valid: PASS
```

This proves HAProxy 2.8.16 accepts the required backend TLS, CA verification, SNI, tcp-check connect, tcp-check send, and tcp-check expect grammar. The production HAProxy configuration remains untouched and no reload/restart occurred.

Result:

```
Phase 8C.13 HAProxy offline TLS/tcp-check syntax validation: PASS
```

The dedicated least-privilege HAProxy health-check ACL identity was then deployed successfully on all three Valkey nodes.

Identity:

```
username = haproxy-health
password = separate randomly generated health-check credential
credential storage = /root/lorawan-secrets/valkey-haproxy-health.txt on ulc-03 during commissioning
```

The ACL is persisted symmetrically in /etc/valkey/valkey.conf on ulc-01, ulc-02, and ulc-03 and was also applied at runtime with ACL SETUSER, without restarting Valkey or changing replication topology.

Effective ACL policy:

```
on
-@all
+ping
+info
```

Observed validation on every node:

```
PING -> PASS
INFO replication -> PASS
GET -> NOPERM / denied
SET -> NOPERM / denied
```

Role detection through the dedicated account also matched the commissioned topology:

```
ulc-01 role=slave
ulc-02 role=slave
ulc-03 role=master
least-privilege role detection: PASS
```

Backups were created before each persistent ACL change. The main Valkey application/replication password was not reused for HAProxy health checks and no production HAProxy configuration was changed.

Result:

```
Phase 8C.13 dedicated HAProxy health-check user: PASS
```

The dedicated HAProxy health credential was then staged successfully on both HAProxy nodes under /etc/haproxy/secrets/valkey-health.env, with directory 0750 root:haproxy and file 0640 root:haproxy. The haproxy service account can read this dedicated credential and /etc/haproxy/ca/valkey-ca.crt, while /etc/lorawan-pki/valkey/server.key remains inaccessible. Secret equality between ulc-03's root-only source and both HAProxy copies was verified by SHA-256 comparison without printing the credential.

Observed checkpoint:

```
ulc-01 health credential access: PASS
ulc-01 CA access: PASS
ulc-01 Valkey private-key isolation: PASS
ulc-01 credential equality: PASS
ulc-02 health credential access: PASS
ulc-02 CA access: PASS
ulc-02 Valkey private-key isolation: PASS
ulc-02 credential equality: PASS
```

The direct authenticated role queries and offline HAProxy role-check validation then completed successfully from both HAProxy nodes. Using only the dedicated haproxy-health identity, both ulc-01 and ulc-02 independently observed the same Valkey topology:

```
ulc-01 role=slave
ulc-02 role=slave
ulc-03 role=master
```

The complete HAProxy 2.8 check grammar also validated offline on both nodes:

```
TLS to backend + CA verification
AUTH haproxy-health <dedicated credential>
expect +OK
INFO replication
expect role:master
```

The credential was supplied through VALKEY_HAPROXY_HEALTH_PASSWORD and was verified absent from the temporary HAProxy config text. The production /etc/haproxy/haproxy.cfg remained valid and unchanged, port 16379 remained free on both HAProxy nodes, and no reload/restart occurred.

Result:

```
Phase 8C.13 authenticated HAProxy role-check preflight: PASS
```

Production design decision: the :16379 frontend will remain TCP/TLS passthrough for application traffic, so client TLS terminates at the currently selected Valkey node and the shared certificate identity valkey.internal.lorawan.com continues to be verified end-to-end. Do not add ssl to the production backend server lines, because that would wrap an already TLS-speaking client stream in a second TLS layer. The health check itself will still use tcp-check connect ssl with CA verification and SNI. A systemd EnvironmentFile=/etc/haproxy/secrets/valkey-health.env drop-in is required so HAProxy can resolve % [env(VALKEY_HAPROXY_HEALTH_PASSWORD)] without storing the secret in haproxy.cfg. Because this environment must be present in the HAProxy master process, commission the two HAProxy nodes sequentially and restart each one only after validating its candidate config, leaving the other HAProxy node available during the brief restart.

The first production :16379 deployment was then attempted on ulc-01 only. Safety checks, backup creation, candidate generation, systemd EnvironmentFile integration, candidate validation, HAProxy restart, service state, master-process environment inheritance, production config validation, and the new 10.104.0.2:16379 listener all passed. Existing PostgreSQL (15432/15433) and MQTT (8883) listeners remained present.

The first functional test exposed a routing defect:

```
PING=PONG
routed role=slave
FAIL: ulc-01 HAProxy did not route to writable primary
```

So TLS passthrough itself is working, but the production health-check behavior is not yet excluding replicas. ulc-02 has NOT yet received the production :16379 configuration and must remain untouched until ulc-01 is corrected.

Current safe state:

```
ulc-01 :16379 listener = active
ulc-01 TLS passthrough = PASS
ulc-01 writable-primary selection = FAIL (routed to replica)
ulc-02 production :16379 deployment = NOT STARTED
```

The exact production block on ulc-01 was then inspected and matched the intended design: frontend `valkey_primary` binds `10.104.0.2:16379`; the backend uses option `tcp-check`; TLS health connections use SNI `valkey.internal.lorawan.com`; AUTH uses the dedicated environment-backed `haproxy-health` credential; INFO replication is followed by expect string `role:master`; and all three Valkey nodes are listed as checked backends. The production block therefore contains no obvious backend-order or plaintext/TLS configuration mistake.

A likely remaining explanation was HAProxy initial server state: checked servers may initially be available until failed health checks mark replicas DOWN. A delayed read-only re-test was therefore started after confirming the real Valkey roles remained `ulc-01=slave`, `ulc-02=slave`, `ulc-03=master` and waiting 15 seconds for health checks to settle.

The delayed test then terminated at the very first `:16379` role query before printing any routed role. Because the wrapper used `set -e` and the `valkey-cli` call was inside command substitution, any non-zero client result aborted the script before the HAProxy log section could run. This new evidence means the endpoint may have transitioned from the initial unsafe state (replica considered eligible) to a state with no eligible backend at all. That would point to a runtime health-check failure affecting every backend, including the real primary, rather than only an initial-state window. This must be diagnosed explicitly; do not assume `init-state fully-down` is the fix yet.

The first runtime diagnostic checkpoint confirmed that HAProxy itself remains healthy and the production listener still exists after the role checks have had time to settle:

```
haproxy=active
LISTEN 10.104.0.2:16379
```

The smaller runtime diagnostic then identified the actual health-check failure. `10.104.0.2:16379` remained listening and HAProxy remained active, but the client received `SSL_connect failed: unexpected eof while reading`. HAProxy logs showed all three Valkey backends becoming DOWN at exactly the same point: `Layer7 timeout ... at step 3 of tcp-check (expect string '+OK')`. After that HAProxy reported backend `'valkey_primary_backend'` has no server available, and later frontend traffic was logged against `<NOSRV>`. Therefore the failure occurs before INFO replication and before the `role:master` matcher; HAProxy is not receiving the `+OK` response to its AUTH command.

The production AUTH rule currently uses `tcp-check send-lf "AUTH haproxy-health %[env{VALKEY_HAPROXY_HEALTH_PASSWORD}]\r"`. HAProxy's documented Redis role-check examples use exact protocol sends ending in `\r\n`, such as `tcp-check send info\ replication\r\n`. The next safe proof is a temporary loopback-only HAProxy instance on ulc-01 using the same dedicated credential but rendering an exact static `tcp-check send AUTH\ haproxy-health\ <credential>\r\n` into a mode-0600 temporary config. This must not touch `/etc/haproxy/haproxy.cfg` or the systemd service. If that temporary check marks the replicas DOWN at `role:master` and ulc-03 UP, the root cause is the production `send-lf AUTH` framing rather than TLS, ACL permissions, or role detection. Destroy the temporary config/process after the test, keep ulc-02 production deployment untouched, and only then correct the production check.

A temporary loopback-only HAProxy proof was then run on ulc-01 using exact CRLF sends for both AUTH and INFO replication. The temporary configuration validated successfully, started successfully, and was protected mode 0600. The first functional request was issued only about five seconds after startup and still returned a replica (`role:slave`). This result does not yet disprove the exact-CRLF correction because the temporary backend used `timeout check 6s`: a replica can remain initially eligible while its first `role:master` expectation is still waiting to time

out. The proof therefore ended too early to distinguish framing failure from the initial health-check convergence window.

The settled exact-CRLF proof then passed. After 12 seconds, ulc-01 and ulc-02 were both marked DOWN specifically at step 5, expect string 'role:master', while ulc-03 remained eligible. Ten consecutive requests through the temporary loopback frontend all returned role:master, with master_routes=10, non_master_routes=0, and errors=0. This confirms the health-check protocol itself works when AUTH and INFO replication are sent with exact CRLF framing. It also confirms the initial unsafe replica routing was caused by the health-check convergence window: checked backends were initially eligible before their first role check completed.

Production correction definitely requires replacing the broken AUTH/INFO framing with exact CRLF and using a bounded role matcher. The isolated min-recv proof then passed on HAProxy 2.8.16: tcp-check expect min-recv 64 string role:master marked ulc-01 DOWN in 5 ms and ulc-02 DOWN in 14 ms with Layer7 invalid response, while ulc-03 remained eligible. After only a two-second wait, ten consecutive requests returned role:master, with master_routes=10, non_master_routes=0, and errors=0. This eliminates the earlier six-second convergence delay for replicas.

Validated HAProxy 2.8 production behavior is therefore: TLS health connection with SNI and CA verification, exact CRLF for AUTH and INFO replication, min-recv 64 on the role:master expectation, and the dedicated haproxy-health ACL identity. Do not use init-state fully-down on these 2.8 nodes. Apply the correction to ulc-01 only first, validate and reload HAProxy, wait briefly for checks to converge, and require repeated master-only routing before deploying the same corrected block to ulc-02.

The first ulc-01 production correction attempt did not reach validation or reload. Its line-aware Python rewrite succeeded in locating all three target rules, but it wrote the HAProxy escape sequences as doubled backslashes (\\r\\n) rather than the required single escaped CRLF tokens (\r\n). The immediately following literal assertions therefore failed and the wrapper exited before haproxy -c or systemctl reload haproxy. A second normalization pass corrected only those two send-lf lines, preserved min-recv 64, and passed exact-line assertions before reload.

The final ulc-01 production correction is now validated and live. The production rules are tcp-check send-lf "AUTH haproxy-health %[env(VALKEY_HAPROXY_HEALTH_PASSWORD)]\r\n", tcp-check send-lf "INFO replication\r\n", and tcp-check expect min-recv 64 string role:master. haproxy -c passed, HAProxy reloaded successfully, and the existing PostgreSQL 15432/15433, MQTT 8883, and Valkey 16379 listeners all remained present. Health logs marked ulc-01 and ulc-02 DOWN at the role:master matcher in 6 ms and 5 ms respectively, while ulc-03 remained eligible. Ten consecutive requests through 10.104.0.2:16379 all returned role:master (master_routes=10, non_master_routes=0, errors=0). A write through the ulc-01 HAProxy endpoint succeeded and the value was verified directly on the current ulc-03 primary, then the test key was removed. Therefore ulc-01 production writable-primary routing is PASS.

The same proven writable-primary configuration is now deployed on ulc-02 at 10.104.0.4:16379. The dedicated health credential and CA were already present and readable only by the HAProxy service as intended; the Valkey private key remained inaccessible. The candidate configuration passed exact AUTH/INFO CRLF assertions, min-recv 64, secret-exclusion, and haproxy -c validation. The systemd EnvironmentFile drop-in was installed, HAProxy restarted successfully so the master process inherited the health credential, and existing PostgreSQL 15432/15433 plus local Mosquitto 8884 listeners remained present. Health logs rejected ulc-01 and ulc-02 as replicas in 7 ms and 6 ms respectively. Ten consecutive requests through 10.104.0.4:16379 all returned role:master with zero errors. Both HAProxy endpoints then independently reported role:master; a write through ulc-02 was read successfully through ulc-01 and verified directly on ulc-03 before cleanup. Phase 8C.13 dual HAProxy writable-primary routing is PASS.

Phase 8C.14 completed the end-to-end automatic failover proof. The starting primary was ulc-03 (10.104.0.8) with healthy Sentinel quorum and both HAProxy :16379 endpoints routing to the writable master. A pre-failover key was written through ulc-01 HAProxy and read through ulc-02. Only the ulc-03 Valkey service was then stopped; its Sentinel remained active. Sentinel elected ulc-02 (10.104.0.4) as the new primary after seven polling attempts,

and all three Sentinels agreed on that new primary. Direct verification showed ulc-02 as `role:master` with ulc-01 online as a replica.

Both HAProxy endpoints automatically followed the promoted primary with no HAProxy configuration change or reload. Each endpoint was briefly unavailable for one polling attempt during convergence, then routed to `role:master`; ten consecutive post-failover requests through each endpoint were master-only with zero errors. The pre-failover value survived, a new write through ulc-02 HAProxy succeeded, the new value was read through ulc-01 HAProxy, and it was verified directly on the new ulc-02 primary. ulc-03 Valkey was then restarted; it transitioned from its prior master state to `role:slave`, followed ulc-02, and reached `master_link_status:up` on attempt 9. Final topology was ulc-02 primary with ulc-01 and ulc-03 online as replicas (`connected_slaves:2`). Sentinel quorum remained healthy, the test key was removed, and no manual failback was performed.

Phase 8C Valkey HA is therefore complete: TLS-only Valkey on three nodes, authenticated replication, three TLS Sentinels with quorum 2, automatic promotion and replica recovery, dual HAProxy writable-primary endpoints on 10.104.0.2:16379 and 10.104.0.4:16379, verified TLS health checks using the least-privilege haproxy-health identity, exact CRLF AUTH/INFO replication framing, `min-recv 64` master-role rejection, master-only routing, and successful automatic HAProxy convergence after Sentinel failover. Current primary is ulc-02 (10.104.0.4). Keep the Sentinel-elected primary in place; do not manually fail back.

Acceptance criteria for the completed Valkey HA layer:

```
Three Valkey nodes installed
One primary
Two replicas
Three Sentinel members
Automatic promotion tested
HAProxy follows writable primary
```

Deployment documentation rule

All executed commands, configuration decisions, validation output, failures, and corrections must be recorded in this MDS before repository commit.

The operator performs the final Git commit and push after review.

Next: [09-chirpstack-cloud-cluster.md](#).

Current gateway-facing MQTT authentication correction - 2026-08-28

The target design earlier in this manual shows the gateway-facing Mosquitto backend :8884 with `require_certificate true` and `use_identity_as_username true`. Later live inventory is authoritative for the current runtime: both brokers presently expose :8884 with `require_certificate=false`, `allow_anonymous=false`, no `use_identity_as_username`, and no gateway-specific ACL/plugin. Treat the earlier mTLS block as the **target configuration**, not proof that gateway client-certificate authentication is already commissioned.

The cloud gateway mTLS boundary is therefore still open even though the broker TLS/failover foundation is healthy. Before public `mqtt.<DOMAIN>:8883` is activated, commission the gateway authentication boundary canary-first on ulc-01 then ulc-02: issue one cloud MQTT `clientAuth` certificate whose CN is the authoritative Gateway EUI `0016c001f139a1cb`, add an exact-EUI ACL for the `as923/gateway/0016c001f139a1cb/... event/state/command` hierarchy, enable client-certificate authentication on the gateway-facing :8884 listener, and prove positive mTLS plus cross-EUI/no-certificate denial through the existing anchor :8883 TLS-passthrough path. This work does not require the final public FQDN or Reserved IPv4 and should be completed before provider public activation.

Gateway MQTT cloud client certificate issuance PASS - 2026-08-28

The real physical Gateway EUI 0016c001f139a1cb now has a dedicated cloud MQTT client identity issued on ulc-03 from the commissioned internal CA. Issuance directory: /root/lorawan-pg-ca/gateway-0016c001f139a1cb-issuance-20260828T015149Z; protected transfer bundle is its transfer/ subdirectory. The certificate subject is CN = 0016c001f139a1cb, issuer CN = LoRaWAN PostgreSQL Internal CA, serial D8732205912F3C3AC56E0A01E1E10583, validity 2026-08-28 01:51:51Z through 2027-09-29 01:51:51Z, SHA-256 fingerprint 82:C6:9A:D7:12:5D:8C:45:F3:8F:BA:AB:F9:6E:7E:3B:41:F4:BE:78:95:FE:16:05:FD:58:2E:09:9D:F3:0A:ED, and certificate SHA-256 f348cef6e280dff82722ff908cc96e694faf183eec1dffc55a7abc690cb472d8.

The certificate chains successfully for sslclient, is rejected for sslserver, and its public key matches the RSA-3072 private key. The issuing CA SHA-256 remains 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb; the existing CA serial-file SHA-256 remained byte-identical at 50df8c462ef9465ab9198284fa1234f0cbfa4f33eb9779ce6d50dd23a618463d because issuance used an explicit random serial. GATEWAY_MQTT_CLIENT_CERT_ISSUANCE=PASS. No Mosquitto, HAProxy, DNS, firewall, or gateway runtime was changed by issuance.

Next boundary: canary-harden ulc-01 gateway-facing Mosquitto :8884 only to require client certificates, map certificate CN to MQTT username, and enforce an exact-EUI as923 gateway ACL. Preserve ChirpStack :8885 and Node-RED :8886. After ulc-01 passes direct mTLS verification with the new gateway certificate, repeat the same bounded rollout on ulc-02.

ulc-01 gateway mTLS canary PASS - 2026-08-28

ulc-01:8884 was hardened from server-TLS-only to gateway client-certificate authentication without changing the ChirpStack :8885 listener. The active gateway listener now has require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. The exact Gateway EUI 0016c001f139a1cb is permitted to write only its own as923/.../event/# and state/# topics and read only its own command/# hierarchy. Mosquitto restarted successfully, :8884 and :8885 remained listening, the issued gateway certificate completed TLS and MQTT CONNECT/SUBSCRIBE successfully, and a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015613Z. Next boundary: apply the same bounded change to ulc-02 and prove the same certificate/ACL behavior there before transferring the cloud certificate bundle to Gateway OS.

ulc-02 gateway mTLS rollout PASS - 2026-08-28

ulc-02:8884 now matches the proven ulc-01 gateway-authentication boundary: require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. Gateway EUI 0016c001f139a1cb is limited to its own AS923 event/state/command hierarchy. Mosquitto restarted successfully; :8884 and ChirpStack :8885 remained listening. The issued gateway certificate completed TLS, MQTT CONNECT, and own-command SUBSCRIBE successfully, while a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015801Z. Therefore both cloud MQTT broker backends now enforce the intended per-gateway mTLS identity. Next boundary: transfer only the three-file cloud certificate bundle (ca.crt, 0016c001f139a1cb.crt, 0016c001f139a1cb.key) to Gateway OS; do not change the working local Mosquitto topology yet.

ulc-02 gateway mTLS state observed - 2026-08-28

Live read-only inspection after the first rollout wrapper showed ulc-02 is already in the intended gateway mTLS state: /etc/mosquitto/conf.d/tls.conf has listener 8884, require_certificate true, allow_anonymous false, use_identity_as_username true, and acl_file /etc/mosquitto/gateway.acl; /etc/mosquitto/gateway.acl exists as 0640 root:mosquitto; per_listener_settings true remains active; and the dedicated ChirpStack listener remains 10.104.0.4:8885 with its own password/ACL files. Therefore do not reapply the mutation. The remaining boundary is a direct positive mTLS/MQTT proof with gateway certificate CN 0016c001f139a1cb plus a no-client-certificate rejection proof.

Both cloud gateway MQTT brokers mTLS acceptance PASS - 2026-08-28

Direct verification from ulc-03 against ulc-02:8884 using the issued gateway identity CN = 0016c001f139a1cb passed TLS, MQTT CONNECT, and subscription to the gateway's own as923/gateway/0016c001f139a1cb/command/# hierarchy. A client without a certificate was rejected. Combined with the earlier ulc-01 canary proof, both gateway-facing Mosquitto backends now enforce per-gateway mTLS and the exact EUI ACL while preserving the dedicated ChirpStack :8885 listeners. The cloud broker-side gateway authentication boundary is therefore complete. Next: transfer only ca.crt, 0016c001f139a1cb.crt, and 0016c001f139a1cb.key from the protected ulc-03 issuance bundle to Gateway OS; do not alter the local loopback broker topology during certificate import.

Supporting runbook: 15-failover-chaos-and-acceptance-testing.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

15. POC Failover and Acceptance Tests

Status: FIRST FAILURE-INJECTION PHASE / DRAFT. Phase 15 starts only after [14b-pre-test-commissioning-gate.md](#) records `PRE_TEST_COMMISSIONING_GATE=PASS`. All required services, Grafana, OpenBao, reviewed Fabric adapters, backups, and the healthy evidence harness must already exist before the first counted fault.

15.0 Hard entry gate

Before creating the first HA- . . . run ID, verify the exact Phase 14B PASS evidence and freeze reference are still current. If a required component is absent, degraded, upgraded, reconfigured, or no longer matches the frozen image/config hashes, **do not inject a fault**. Return to setup, repair/recommission it, take a new Phase 13B snapshot, rerun Phase 14, and issue a new Phase 14B decision.

A missing Fabric adapter or required gateway-integrity implementation is therefore a **setup blocker**, not an acceptable surprise discovered midway through counted testing.

This file proves the **small HA model**, not production capacity.

Use one staging gateway/device. Inject one failure at a time and restore full health before the next test.

For every run create a unique identifier such as:

```
HA-YYYYMMDD-<TEST>-R<NUMBER>
```

Record all timestamps in UTC.

Define the main LoRaWAN recovery time consistently:

```
RTO start = timestamp immediately before the fault is injected
RTO end   = timestamp when the first NEW post-fault staging-device uplink is
            accepted through the surviving ChirpStack path
```

Record database-role recovery, MQTT reconnect, UI/API recovery, and outbox recovery separately rather than mixing them into one number.

Standard test loop

Use this sequence for **every** failure scenario:

1. create the run ID and UTC start record;
2. capture the full healthy baseline below;

3. send one pre-fault real uplink and record its test_sequence/event key;
4. inject exactly one named failure;
5. record the exact fault timestamp and action;
6. observe quorum/role changes without manually forcing a second failure;
7. send fresh post-fault uplinks until the first one is accepted;
8. record RTO and any duplicates/gaps/reconnects;
9. restore the failed process/member/host;
10. wait for **3/3 quorum/replica redundancy where applicable** and confirm no pending repair;
11. send one post-restore uplink;
12. capture memory/OOM evidence and sanitized logs;
13. close the run only after the system is back at the normal baseline.

Do not continue to the next test from a degraded state. A second failure while a quorum group is still at 2/3 changes the experiment into a two-failure scenario.

15.1 Baseline before every failure

Capture:

```
real uplink succeeds
both ChirpStack instances healthy
etcd 3/3
Patroni 1 primary + 2 replicas
both databases present: chirpstack + Timescale-enabled lorawan_telemetry
TimescaleDB extension version recorded; uplinks + measurements hypertables visible
Valkey primary + 2 replicas
3 Sentinels
Mosquitto-1 + Mosquitto-2 healthy
Reserved IPv4 owner recorded
public-ingress failover timer healthy on ha-01/ha-02
both HAProxy anchor listeners pass local health
OpenBao 3/3 healthy/unsealed
Node-RED healthy; latest real telemetry event stored and duplicate-safe
Grafana healthy/read-only; latest reading visible
OpenBao 3/3 healthy/unsealed
Fabric adapter-1 + adapter-2 healthy with one normal confirmed tx already proven
Phase 13B final backup-set ID recorded and off-host
Phase 14 healthy evidence dry run reference recorded
host memory / swap / OOM log
```

Do not inject a failure from an already degraded baseline.

Useful baseline commands include the exact version-specific equivalents of:

```
# every cloud host
free -h
uptime
command -v vmstat >/dev/null && vmstat 1 5 || true
docker stats --no-stream
journalctl -k --since today | grep -Ei 'oom|out of memory|killed process' || true

# etcd administration host - current validated transport baseline
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health

# If etcd transport TLS is deliberately deployed and validated before these tests,
# update this command to the matching tested https endpoints and CA/client credentials.

# PostgreSQL administration host
```

```
patronictl -c <PATRONI_CONFIG> list <PG_SCOPE> --extended
```

```
# PgBouncer on each client host  
psql '<PGBOUNCER_ADMIN_DSN>' -c 'SHOW POOLS;'
```

Also record Valkey/Sentinel, Mosquitto, OpenBao, and Fabric-outbox states using their component manuals. Do not print secrets into the evidence bundle.

15.2 Lose ha-01

Fault injection procedure:

1. confirm ha-02 and ha-03 are healthy and redundancy is 3/3 before the test;
2. deliberately assign the Reserved IPv4 to ha-01, verify public HTTPS/MQTT health, and confirm ha-02 is a healthy standby ingress candidate;
3. record whether ha-01 currently owns the PostgreSQL primary, Valkey primary, active OpenBao role, preferred Mosquitto, or any live outbox lease;
4. from the provider console or the host, power off **ha-01 only** using the approved POC fault method;
5. do not manually promote PostgreSQL, Valkey, or the Reserved IP unless automatic failover itself is the thing being diagnosed;
6. watch the ha-02 public-ingress agent take the etcd lock and reassign the Reserved IPv4;
7. send a fresh staging-device uplink and measure recovery;
8. after evidence is captured, power ha-01 back on and wait for every colocated member/service to rejoin before closing the test. The Reserved IP should remain on ha-02 until an operator deliberately moves it later.

Expected:

```
Reserved IPv4 automatically moves to ha-02  
DNS remains unchanged  
ChirpStack-2 serves  
Mosquitto-2 is available  
etcd remains 2/3  
OpenBao remains 2/3  
Patroni remains/promotes a primary  
Valkey remains/promotes a primary  
adapter-2 remains only when the conditional Fabric execution scope is deployed  
real uplink recovers
```

Record recovery time and memory on ha-02/ha-03.

Restore ha-01 and wait for full replication/quorum before continuing.

15.3 Lose ha-02

Use the same host-loss procedure as 15.2, but first deliberately assign the Reserved IPv4 to **ha-02**, prove ha-01 is a healthy ingress candidate, then power off **ha-02 only**. Record its current roles before failure because the PostgreSQL/Valkey primary may have moved during earlier tests.

Expected: the Reserved IPv4 automatically reassigns to ha-01, DNS remains unchanged, and quorum groups remain at 2/3.

Restore ha-02, wait for full membership/replication, and send a post-restore uplink before continuing.

15.4 Lose ha-03

Before fault injection, prove one telemetry row is committed and identify any existing eligible outbox row. Then power off **ha-03 only**.

Expected:

```
etcd remains 2/3
OpenBao remains 2/3
PostgreSQL remains available on ha-01/ha-02
chirpstack database remains
Timescale-enabled lorawan_telemetry remains
telemetry hypertables remain
fabric_outbox remains
Valkey remains available/promotable
ChirpStack + MQTT continue
existing fabric_outbox remains; deployed adapter-1/2 may continue eligible work
Node-RED A on ha-03 is lost; after fencing, Node-RED B on ha-02 is promoted
fresh post-promotion uplink reaches TimescaleDB exactly once
Grafana pauses
```

This test specifically proves that telemetry/outbox **storage** is no longer a single ha-03 service.

Restore ha-03, wait for PostgreSQL/etcd/Valkey/OpenBao membership to return to 3/3, but **do not automatically start Node-RED A** while Node-RED B is active. Re-stage/synchronize A as the stopped standby, then perform the documented fenced failback as a separate step. Grafana may be restarted on ha-03. Record the Node-RED promotion time and any missing QoS-0 uplinks during the transition; active/passive availability does not imply replay.

15.5 Planned PostgreSQL switchover

Use Patroni to move the primary deliberately.

Expected:

```
HAProxy follows new primary
PgBouncer reconnects server connections
ChirpStack keeps same DB endpoint
Node-RED keeps same DB endpoint
Grafana keeps same DB endpoint
deployed Fabric adapters keep same DB endpoint; otherwise the outbox itself remains available
```

Verify both databases after the switchover. In lorawan_telemetry, also verify the TimescaleDB extension version and query timescaledb_information.hypertables; telemetry.uplinks and telemetry.measurements must still be present as hypertables.

15.6 Unplanned PostgreSQL primary loss

Stop only the current PostgreSQL/Patroni primary process or its host as planned.

Pass when:

```
one replica promotes
only one writable primary exists
clients reconnect automatically
real uplink can be processed
```

```
Timescale-enabled lorawan_telemetry is writable again
Timescale hypertables are queryable on the promoted primary
```

15.7 Valkey primary loss

Stop the current Valkey primary.

Pass when Sentinel promotes a replica and ChirpStack recovers without endpoint edits.

15.8 Preferred Mosquitto loss

Stop **Mosquitto-1 only**, not HAProxy and not the whole ha-01 host. This isolates broker-service failover from host failover.

Before stopping it, verify all HAProxy instances select Mosquitto-1 as preferred and the physical gateway has a healthy cloud bridge.

Pass when:

```
gateway reconnects to backup path
gateway local QoS 1 queue protects bounded uplinks
new real uplink reaches ChirpStack
```

Restore Mosquitto-1, prove HAProxy returns to the intended preferred/backup ordering, and verify the gateway local queue is back at its normal drained state.

15.9 One ChirpStack process loss

Stop ChirpStack-1 only.

Pass when ChirpStack-2 processes a fresh real uplink and the public UI/API path still works through the unchanged Reserved IPv4. A single ChirpStack process loss should normally be handled by HAProxy backend routing without moving the Reserved IP.

15.10 One OpenBao member loss

Stop one OpenBao member.

Pass when:

```
2/3 quorum remains
stable KMS endpoint still signs/verifies
adapter does not need a local fallback key
```

Restore 3/3.

15.11 One Fabric adapter loss

Phase 14B should already have proved the adapter implementation readiness gate in [20-openbao-and-fabric-adapter.md](#).

If the reviewed adapter implementation/image is absent now, **abort the counted Phase 15 suite and return to setup**; do not mark only this one test blocked and continue.

With both workers deployed:

1. create one eligible POC outbox job;
2. record its lease owner/state;
3. stop adapter-1 only;
4. wait for the documented lease expiry/reclaim condition;
5. prove adapter-2 claims/processes the job without simultaneous ownership.

Do not accept simultaneous ownership of one live lease.

15.12 External Fabric outage

Both adapters must already be deployed from the Phase 14B PASS baseline. If they are absent or unhealthy before injection, abort the counted suite and return to setup.

Block only the adapter path to the external Fabric Gateway; do not disturb Node-RED, PostgreSQL, or MQTT when injecting this specific fault.

Send a fresh real **EMU-01 payload-v2** uplink and record its test_sequence/event key.

Expected:

```
ChirpStack continues
Node-RED continues
telemetry row commits
fabric_outbox row commits
outbox waits/accumulates
no false confirmed status
```

Restore Fabric connectivity.

Pass when the adapter reconciles uncertain submissions and the outbox drains without conflicting duplicate state.

15.13 4G outage

Disconnect the test gateway's LTE backhaul without changing RF settings, Gateway EUI, MQTT configuration, or local Mosquitto data. Do not stop the gateway-local broker; the purpose is to test its persistent queue.

Expected:

```
gateway remains running
local Mosquitto queue grows
cloud last-seen becomes stale
no stale downlink is replayed
after LTE returns, queue drains and fresh uplinks resume
```

This is a gateway-buffer test, not the same experiment as the local-VM dissertation WAN interruption.

15.14 2-GiB resource test

During each failure run:

```

free -h
uptime
command -v vmstat >/dev/null && vmstat 1 5 || true
docker stats --no-stream
journalctl -k --since today | grep -Ei 'oom|out of memory|killed process' || true

```

The 2-GiB shared-CPU profile fails if an essential surviving service is OOM-killed, the node becomes unusably swap-bound, or CPU contention prevents the documented functional failover/recovery criteria from completing under the few-sensor workload.

If that happens, resize and repeat the same test. The resize result is part of the POC finding.

15.15 Acceptance matrix

Test	Required POC result
ha-01 loss	Reserved IPv4 moves automatically to ha-02, DNS stays unchanged, ha-02 path carries LoRaWAN, quorum groups survive
ha-02 loss	after deliberately starting with the Reserved IPv4 on ha-02, it moves automatically to ha-01 with DNS unchanged
ha-03 loss	LoRaWAN + PostgreSQL/outbox survive; fenced Node-RED B promotion on ha-02 restores fresh telemetry processing exactly once; Grafana may pause
PG switchover	all DB clients keep same endpoint
PG primary loss	one new primary, no manual DSN change
Valkey primary loss	Sentinel promotion works
Mosquito-1 loss	gateway reconnects to backup service
ChirpStack-1 loss	ChirpStack-2 processes fresh uplink; Reserved IPv4 does not needlessly move
OpenBao member loss	KMS works at 2/3
adapter-1 loss	Required for full-feature PASS: adapter-2 continues eligible work after valid lease recovery; missing reviewed adapter implementation makes the overall full-feature result BLOCKED
external Fabric outage	Required for full-feature PASS: telemetry/outbox commit continues, queued work waits safely, then reconciles/drains after Fabric recovery
LTE outage	local gateway queue drains after recovery
2-GiB resource floor	no essential OOM during accepted tests

15.16 What passing means

Passing means:

the small cloud model demonstrates the future HA architecture's important failure relationships with every required full-feature runtime component included.

A BLOCKED Fabric adapter test is useful partial infrastructure evidence, but it is **not** a final full-feature PASS. The overall POC remains BLOCKED until the reviewed adapter is deployed and its loss/outage/reconciliation tests pass.

It does not mean the same 2-GiB machines should be used for the final deployment.

Next: [16-operations-upgrades-and-scaling.md](#).

Canonical source: deployment/technology-guides/09-valkey-sentinel.md. For any mutable production-status statement, check the dated baseline and verified live state.